Arbiter

University Research Publication & Peer-Review Hub

Instructor: Professor Robert Bailey

Student: Divyashanu Swain

Date: October 14th, 2026

GitHub repository: https://github.com/dswain07/Arbiter-Razor_Page

Project overview

Arbiter is an ASP.NET Core Razor Pages web application for organizing academic research submissions, authors, peer reviews, and research tracks. It supports paper discovery through searchable fields and filters, CRUD workflows, an administration interface, and persistent light/dark appearance settings.

Implementation summary

Requirement	Implementation
Web application	ASP.NET Core Razor Pages (.NET 10)
Data layer	Separate Arbiter.Data Class Library with EF Core
Database	SQLite; five application tables and 40 seeded records
CRUD	Papers, Authors, Reviews; administrative Track management
Search	Paper ID, Title, Abstract; Keywords, Status, Track
Settings	Light and dark themes, applied without restart
Admin	Track administration with optional password/cookie protection
Database scripts	DDL.sql and DML.sql tested in a separate SQLite database

1. Answers to the assignment questions

1.1 Describe a challenge you overcame in this project

One challenge was debugging administrator logout in ASP.NET Core Razor Pages. The initial Logout button submitted its POST request to the current Administration page because the generated form action was empty. After correcting the target, the request still returned HTTP 405 (Method Not Allowed). I used Chrome Developer Tools to inspect the rendered form and network requests, then checked endpoint registration to separate a routing problem from a cookie authentication problem. Testing a POST form directly on the AdminLogout Razor Page confirmed that the OnPostAsync handler and SignOutAsync worked. I adopted a two-step logout flow: the navigation link opens a confirmation page, and its form performs the POST sign-out. This taught me to inspect actual HTTP requests instead of assuming that correct looking Razor markup guarantees correct runtime behavior.

1.2 Describe a technique you learned in this project

I learned to use Entity Framework Core navigation properties, LINQ, and eager loading to work with related data. Arbiter stores Papers, Authors, Reviews, Tracks, and PaperAuthors in separate tables, with foreign keys representing their relationships. For example, Include(p => p.Track) loads a paper's associated research track so the UI can display the track name rather than only a numeric ID. I also built composable LINQ queries for paper searching: users can search by ID or partial text in Title, Abstract, and Keywords, and combine the search with Status and Track filters. This approach keeps filtering in the data-access query instead of manually processing all records after retrieval.

1.3 What would you refine before selling, shipping, or deploying?

Before production deployment, I would introduce complete user accounts and role-based permissions for authors, reviewers, and administrators; currently, only the Administration area has optional password-based protection, while other CRUD pages are available without role restrictions. I would also add a formal review workflow with reviewer assignments, submission ownership, file uploads, and publication approval rules. Security improvements would include login-attempt throttling, stronger identity management, server-side validation, audit logs, HTTPS-only deployment, and secret management. Finally, I would add automated tests, accessibility checks, pagination, structured logging, backup/recovery procedures, and a deployment pipeline. These changes would make Arbiter safer, more scalable, and more suitable for real university use.

1.4 Which technology did you choose (Razor or MVC), and why?

I chose ASP.NET Core Razor Pages because Arbiter is organized around page-based workflows such as Papers/Index, Papers/Create, Papers/Edit, and Papers/Delete. Each page has a .cshtml view and an associated PageModel that handles its GET and POST requests. This organization makes it easy to connect form validation, Entity Framework Core operations, and navigation while keeping the project understandable. MVC would also work, especially for a larger application with more shared controller actions, but Razor Pages provided a simpler fit for the scope and CRUD-heavy structure of Minor Project 3.

2. Graduate requirement: separate Class Library data layer

I created a separate .NET Class Library project named Arbiter.Data and included it in the Arbiter solution alongside the ASP.NET Core web project, Arbiter.Web. The data project contains the entity classes (Paper, Author, Review, Track, PaperAuthor), the ArbiterDbContext, and the Entity Framework Core relationship configuration and migrations.

I added a project reference from Arbiter.Web to Arbiter.Data so the web project could use the entity types and DbContext. In Arbiter.Web/Program.cs, I registered ArbiterDbContext with dependency injection using AddDbContext and configured the SQLite connection string. Razor PageModels then receive the DbContext through constructor injection. I used EF Core migrations to create and evolve the database schema and a DbInitializer to seed related records in dependency order: Tracks, Authors, Papers, PaperAuthors, and Reviews.

Separating the data layer allows the database entities and persistence logic to remain independent of the UI. This makes the solution easier to maintain and supports future reuse in another client, service, or API.

3. Database and script verification

Application table	Seeded records
Tracks	5
Authors	8
Papers	10
PaperAuthors	10
Reviews	7
TOTAL	40

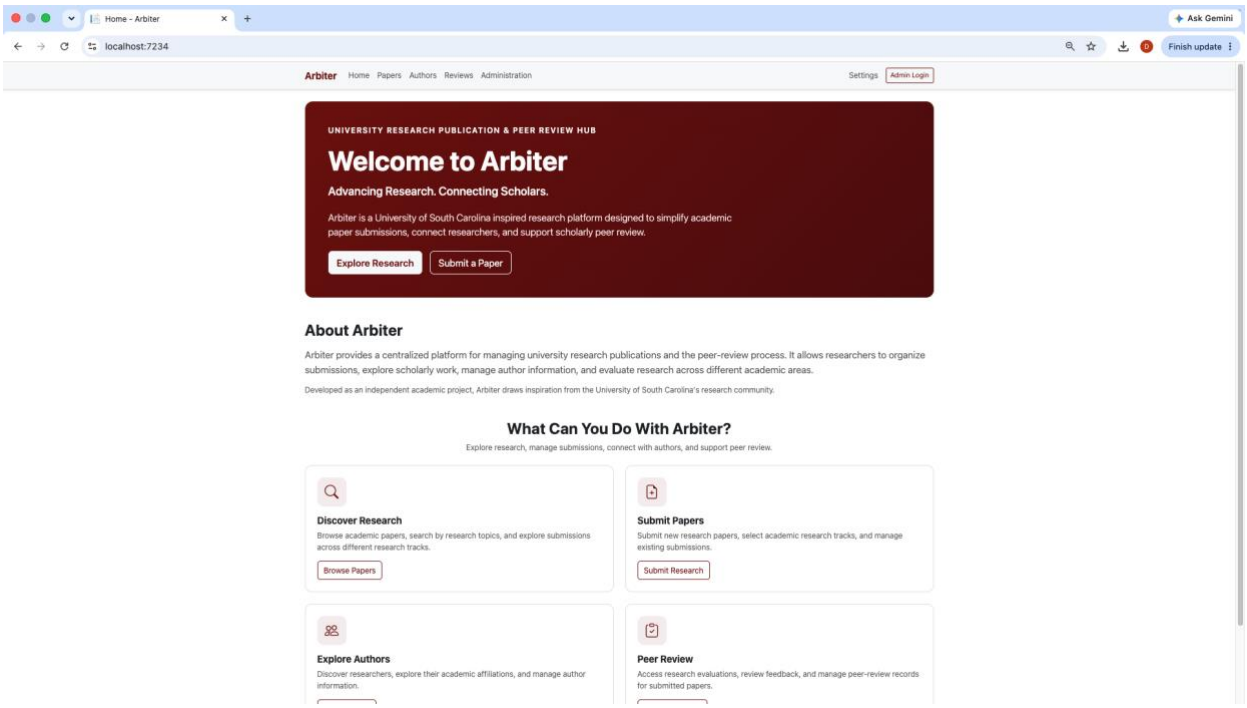
The DDL.sql and DML.sql exports were executed against a separate ArbiterTest.db database. The recreated database contained the five application tables and 40 records, and PRAGMA foreign_key_check returned no violations. This test did not modify the main application database.

4. Light/Dark themes and administration

Arbiter uses Bootstrap 5.3 color modes with the data-bs-theme attribute. The Settings page changes the theme immediately using JavaScript and saves the preference in browser localStorage, so the choice persists across navigation and reloads without restarting the application. The Administration section manages Tracks and quotas and is protected by an optional administrator password with cookie-based authentication and a two-step logout.

5. Screenshot evidence

Homepage



Search

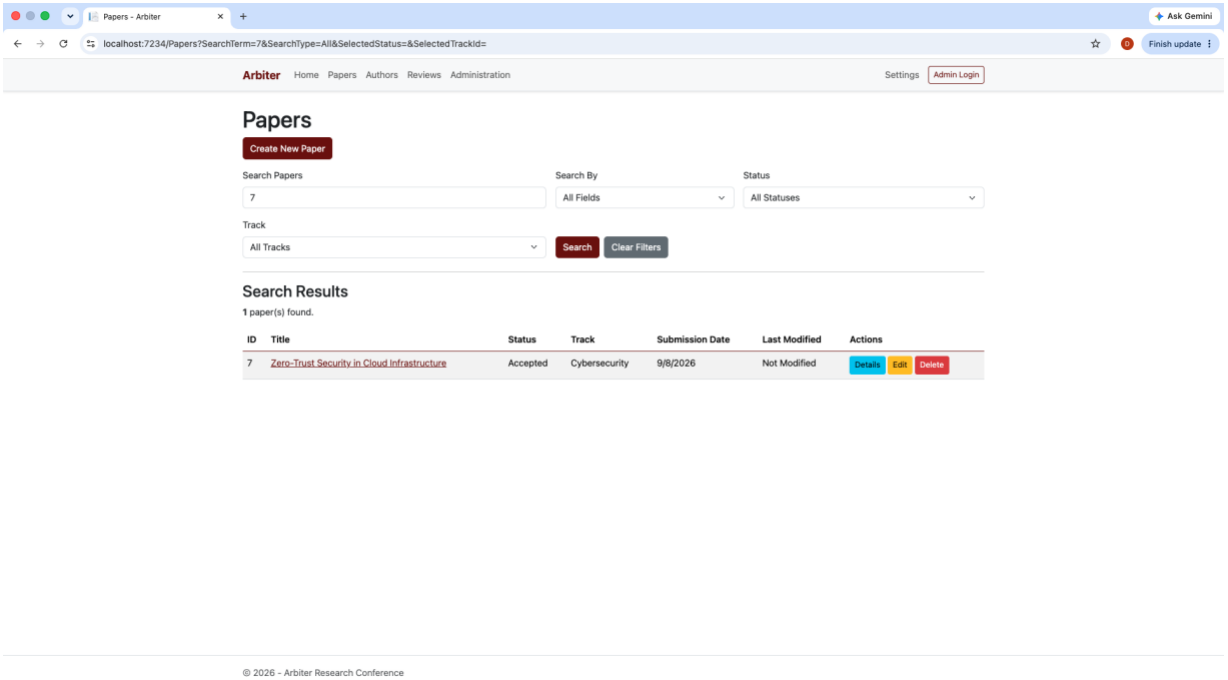


Figure 1. Search by ID

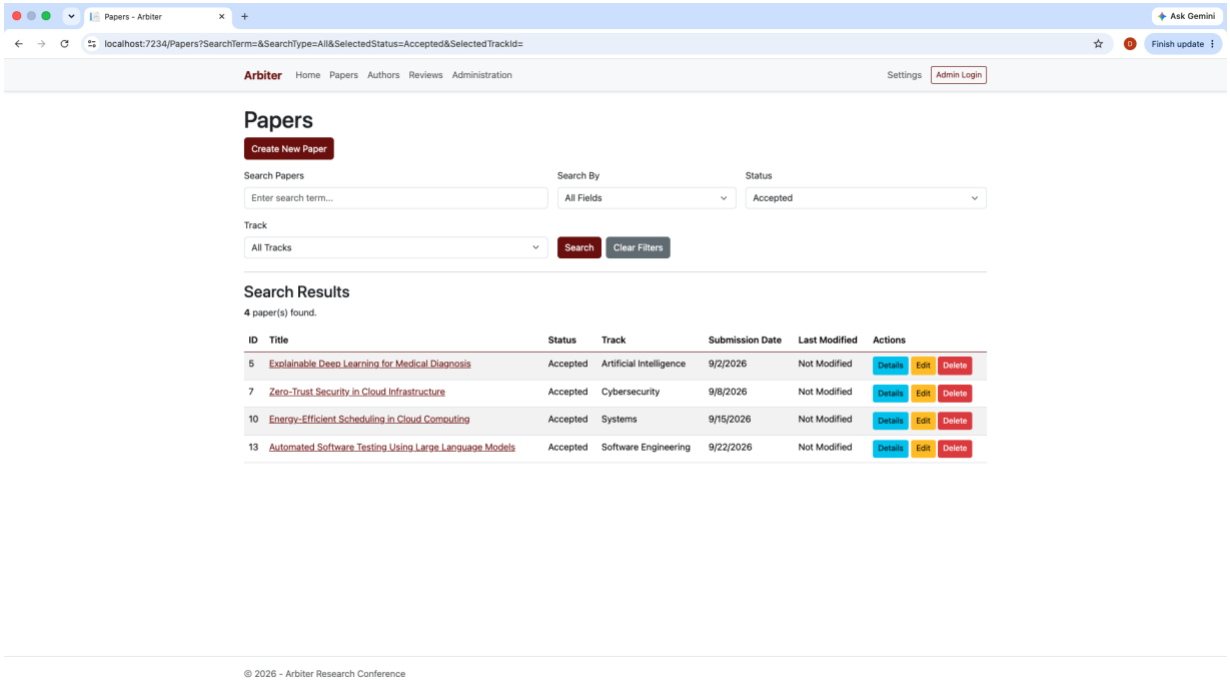


Figure 2. Search by Status

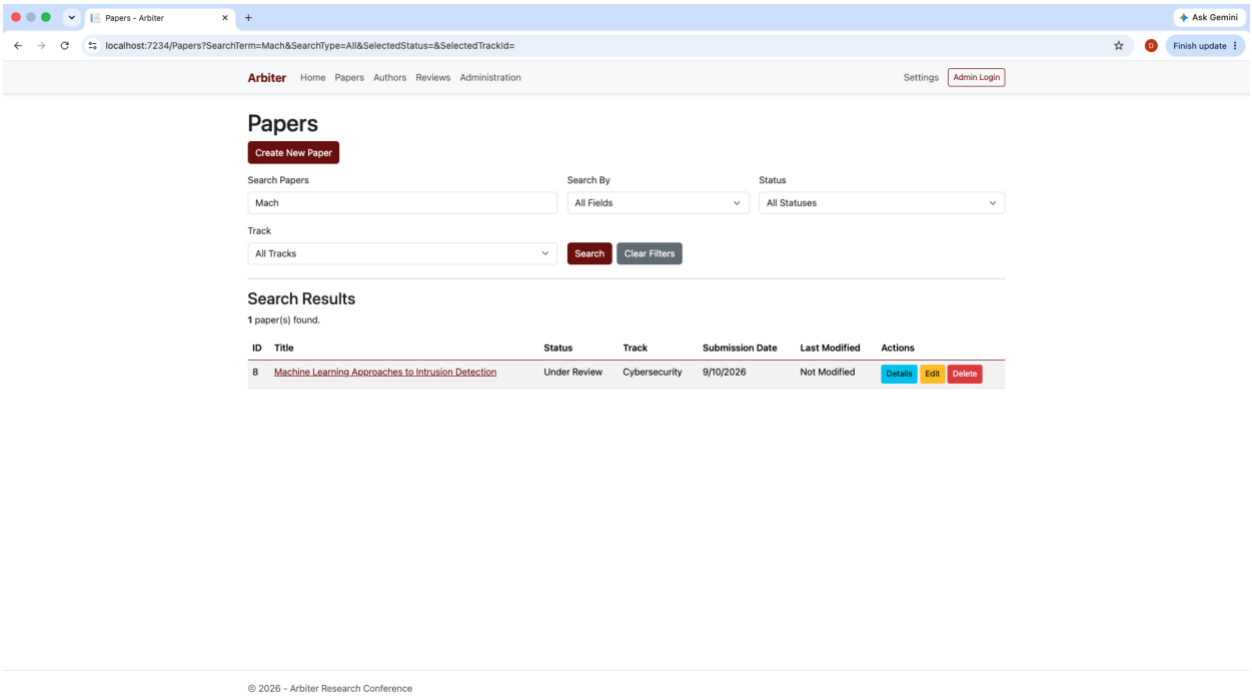


Figure 3. Search by Title

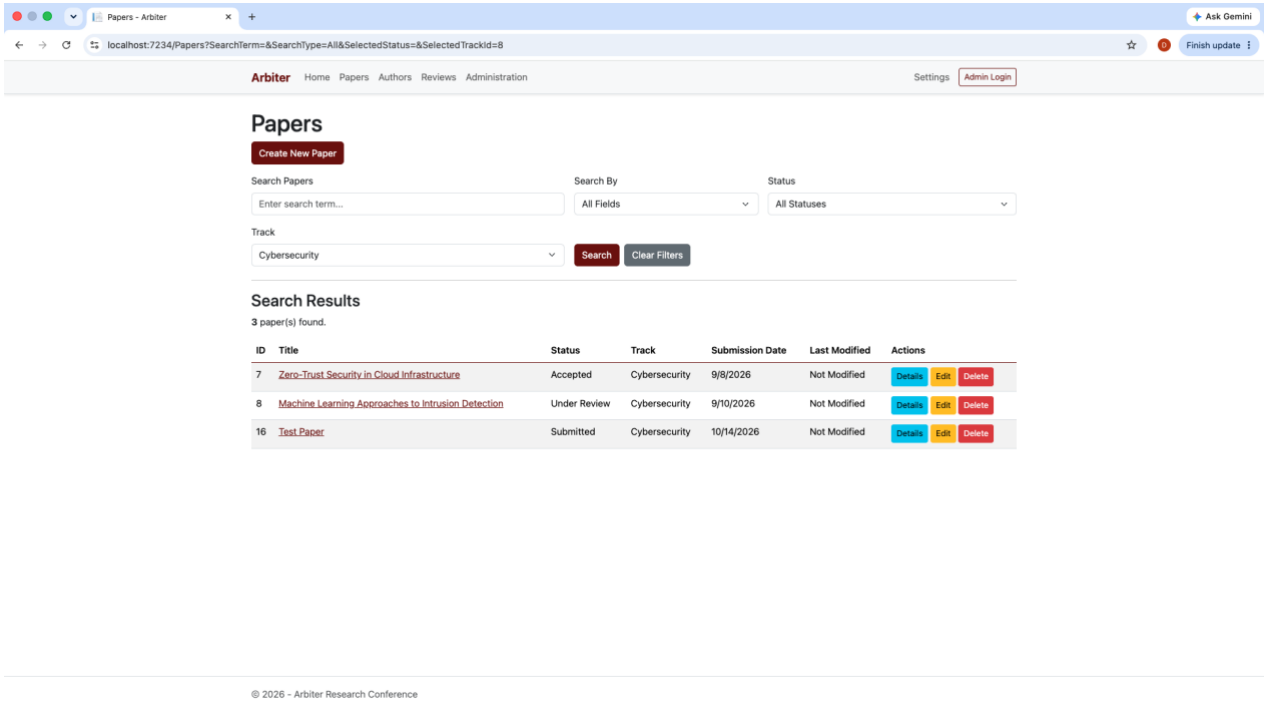


Figure 4. Search by Track

Paper CRUD

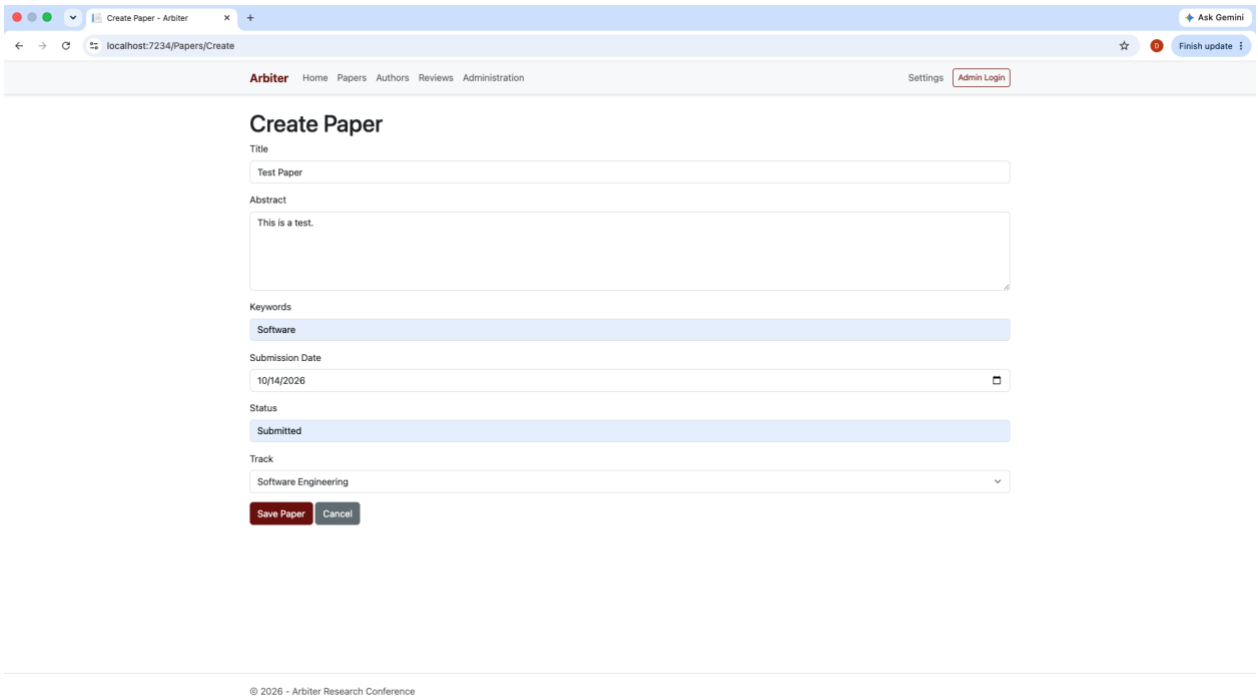


Figure 1. Create Paper

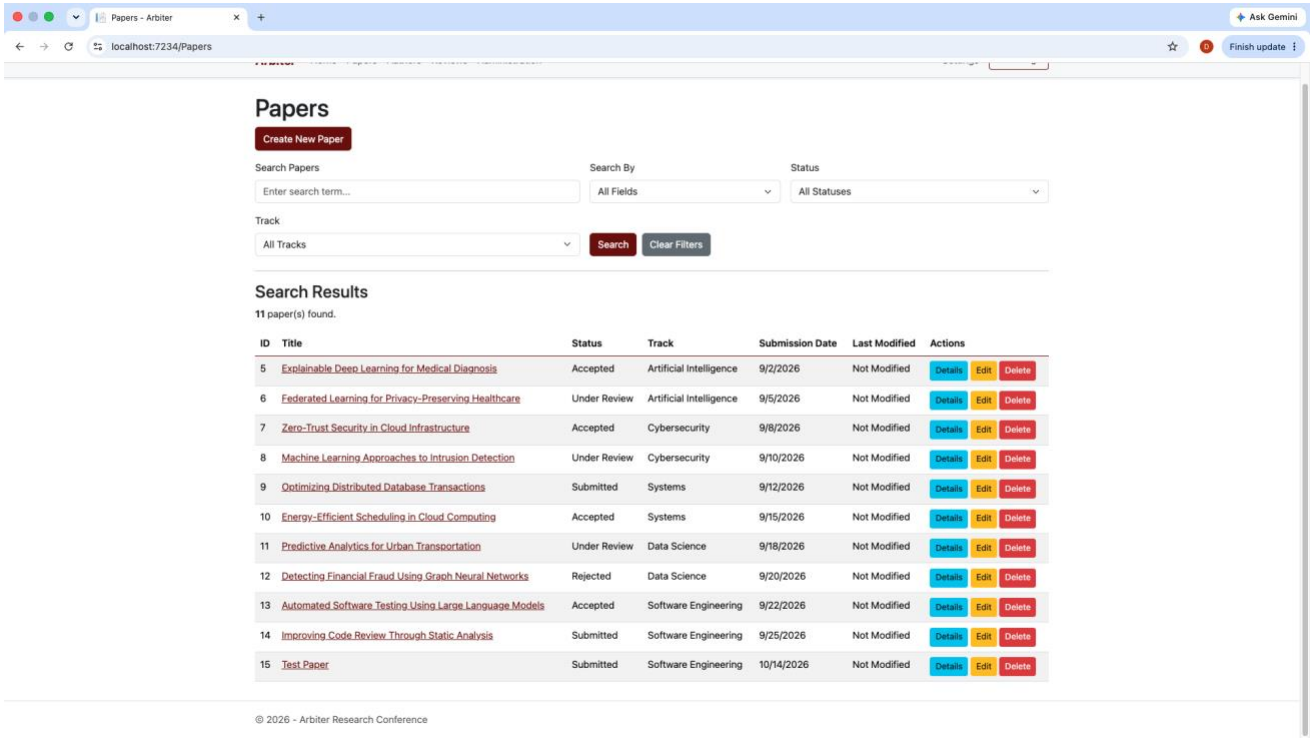


Figure 2. Create Successful

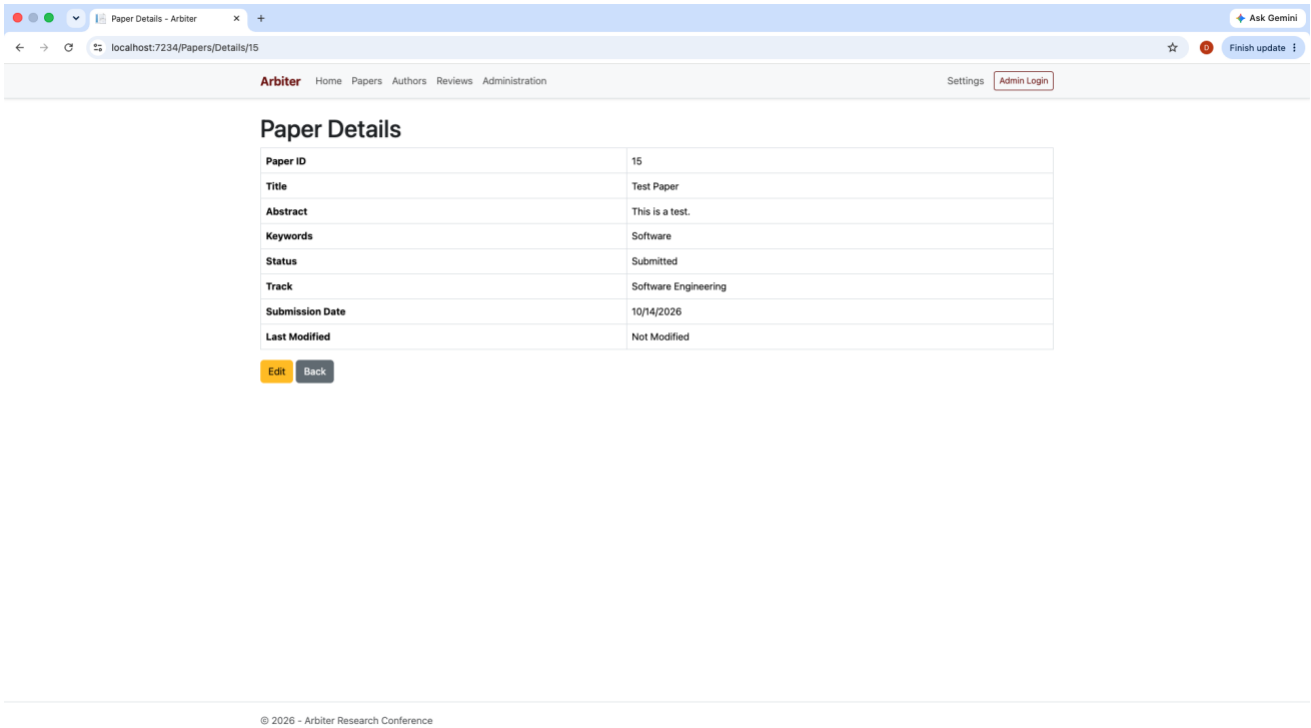


Figure 3. Read Paper

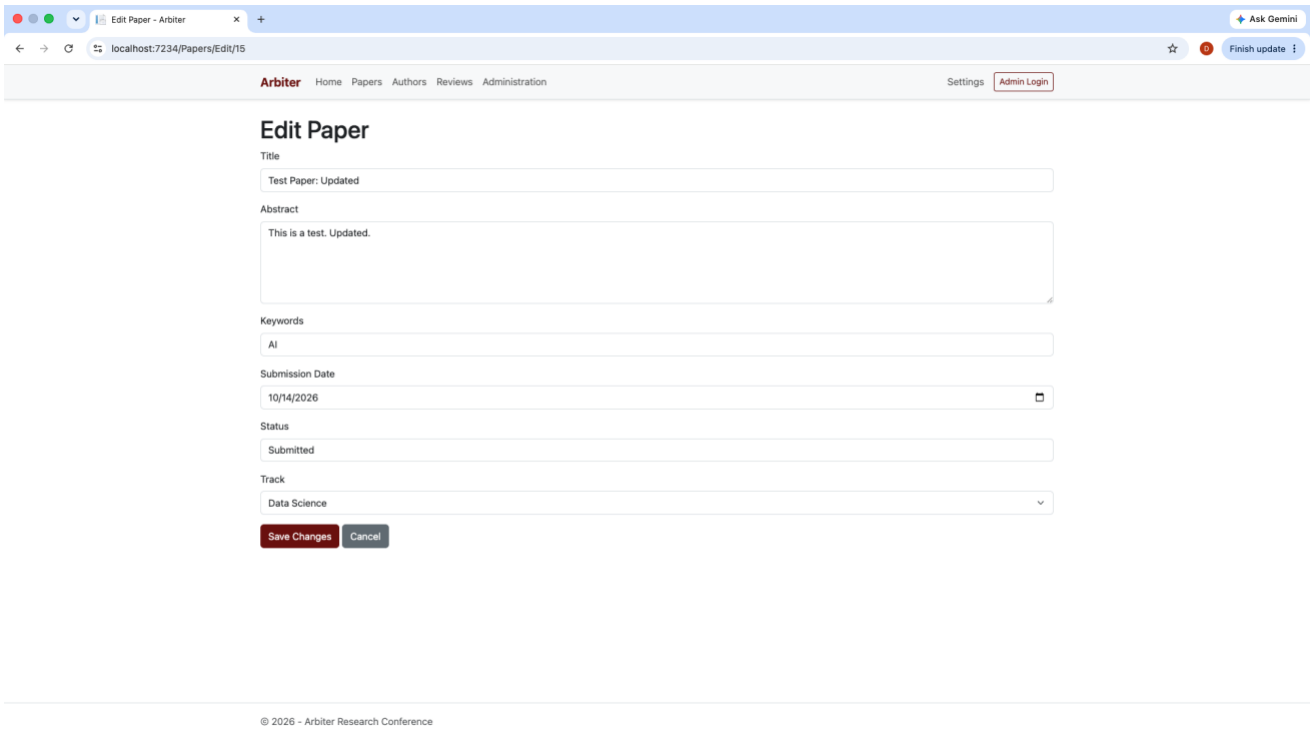


Figure 4. Update Paper

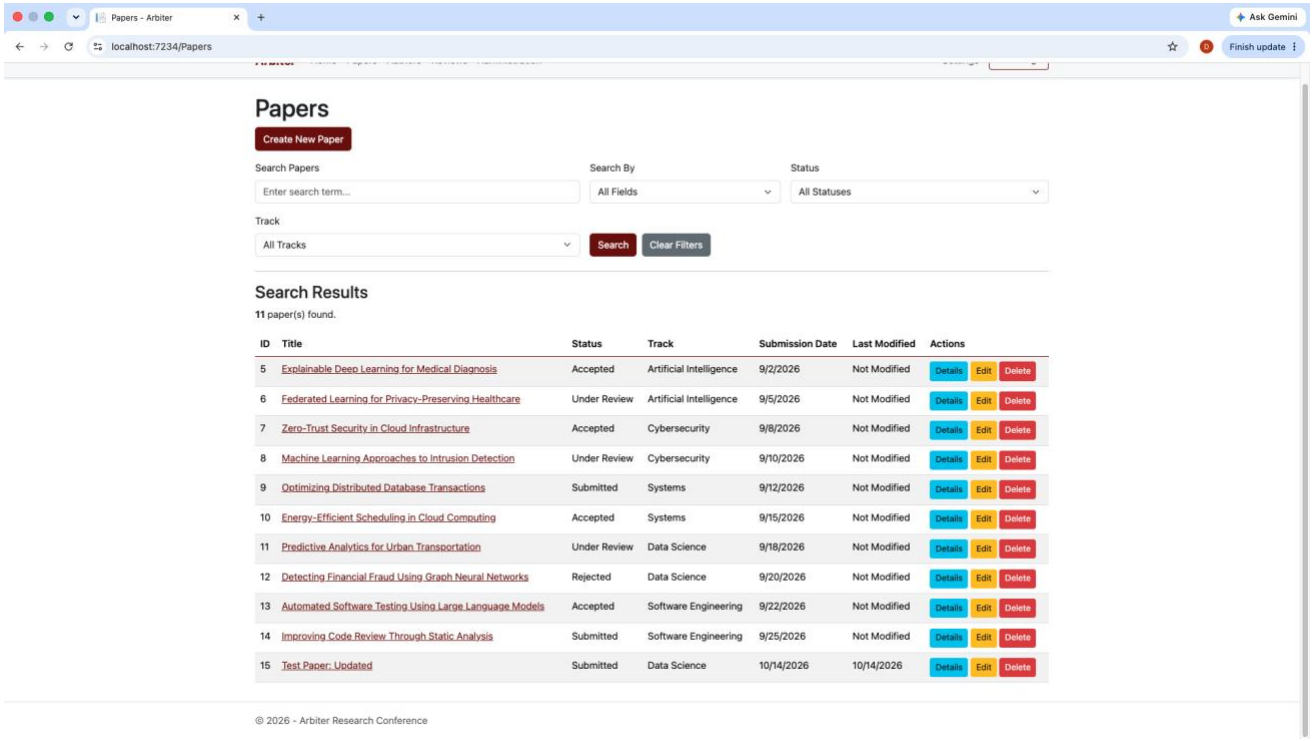


Figure 5. Update Successful

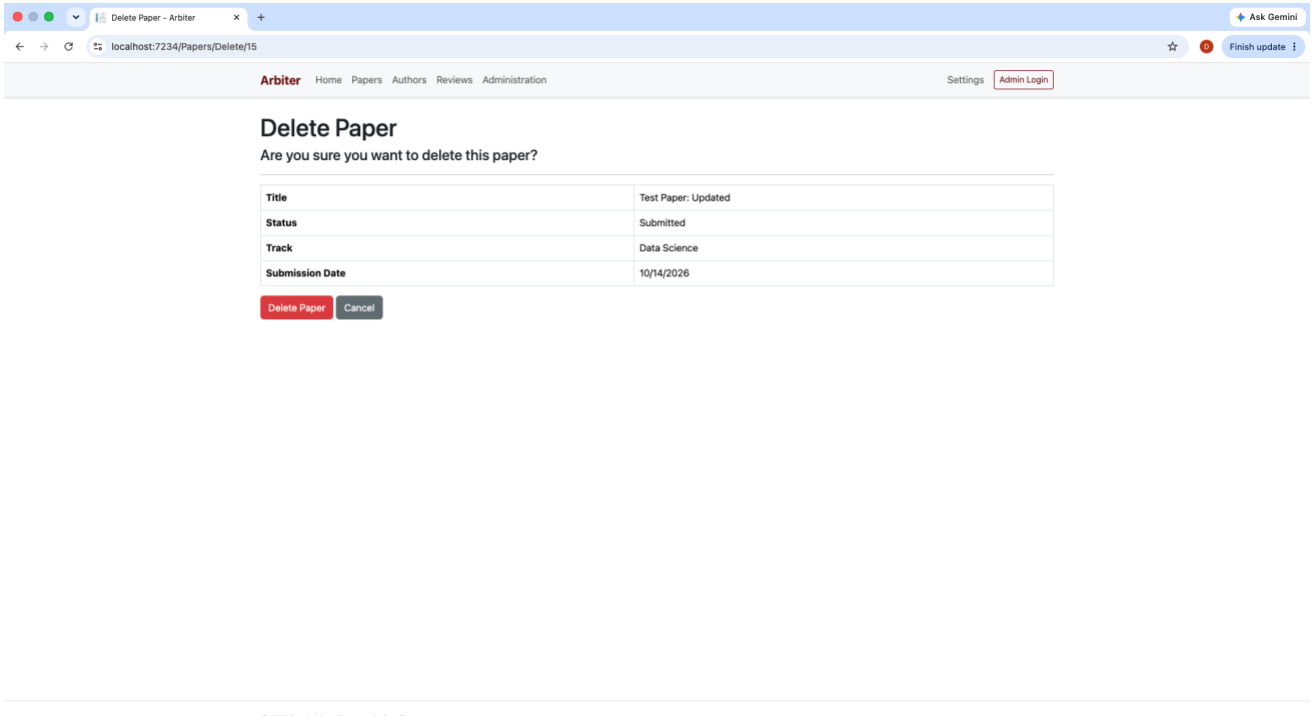


Figure 6. Delete Paper

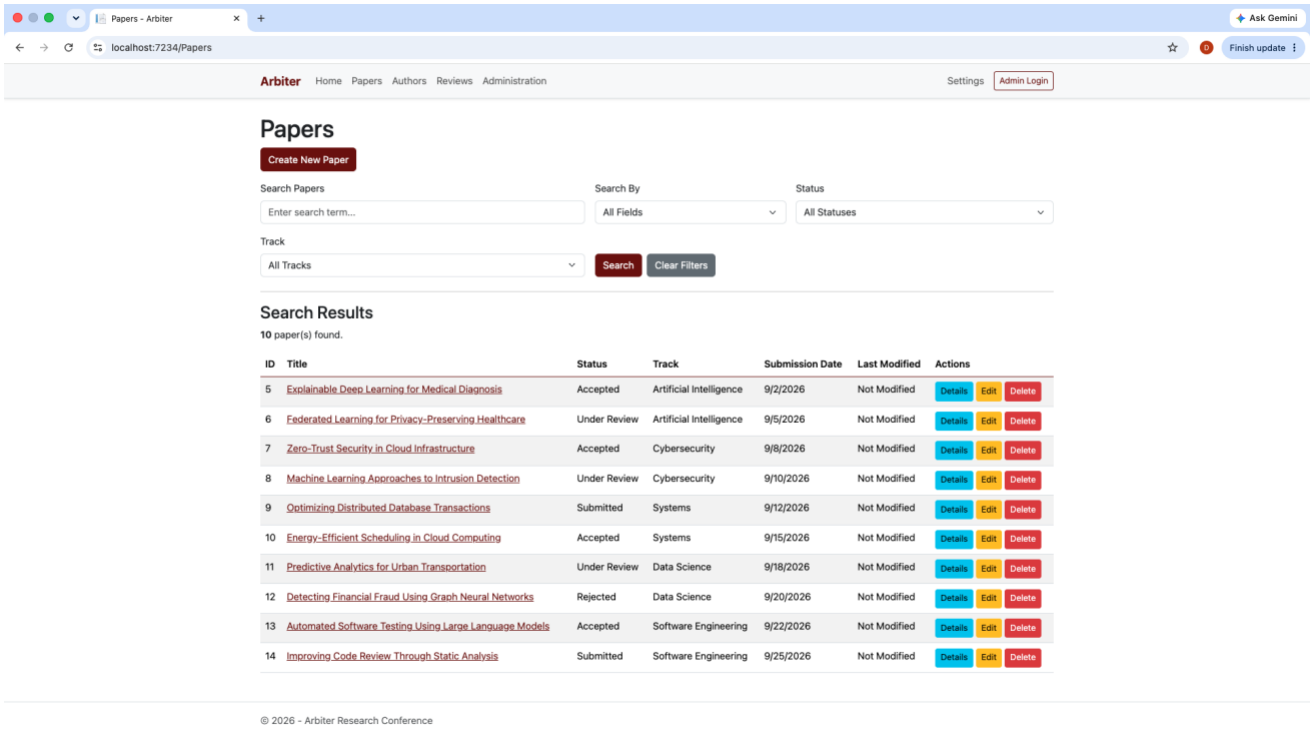


Figure 7. Delete Successful

Author CRUD

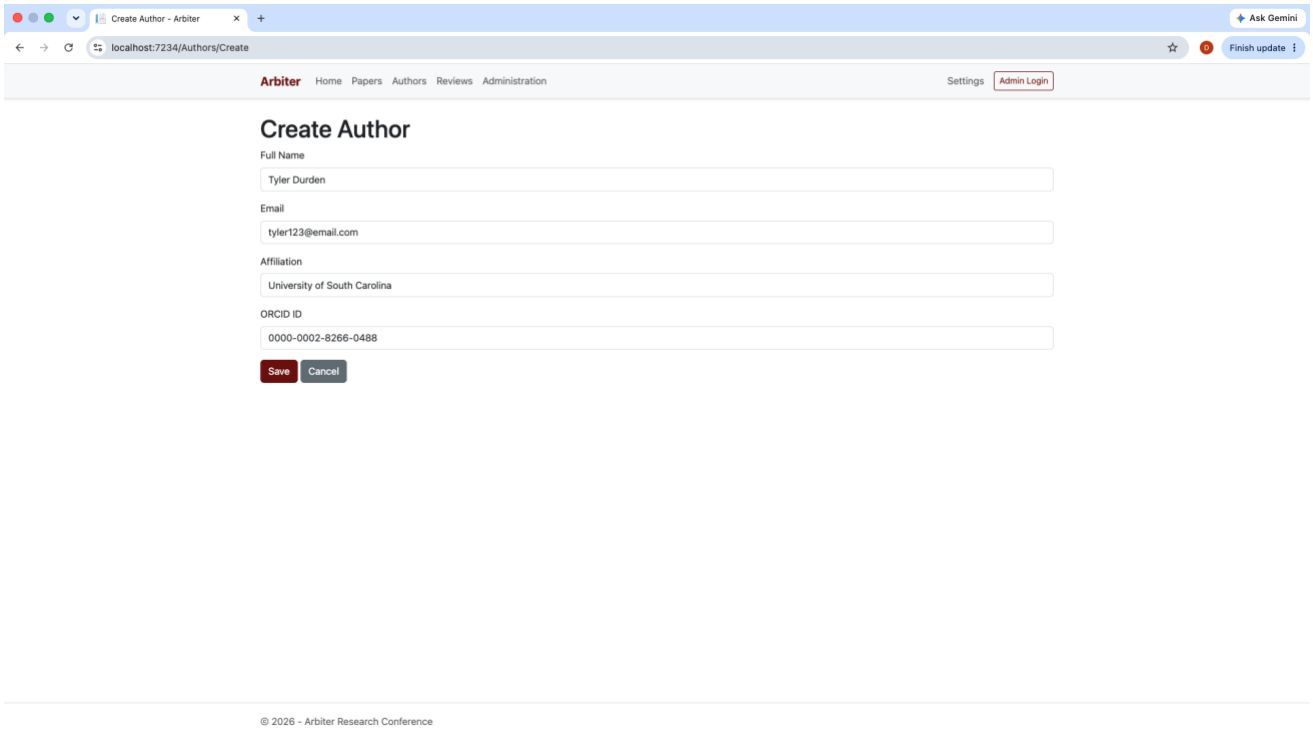


Figure 1. Create Author

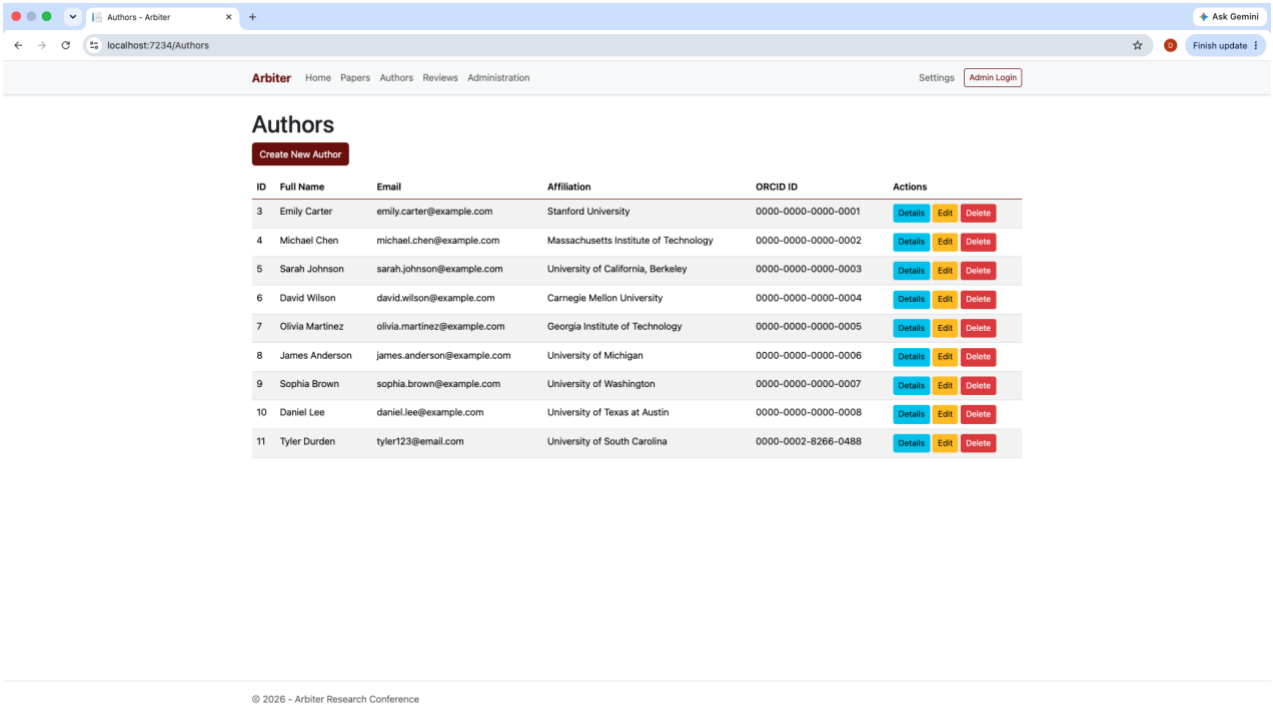


Figure 2. Create Successful

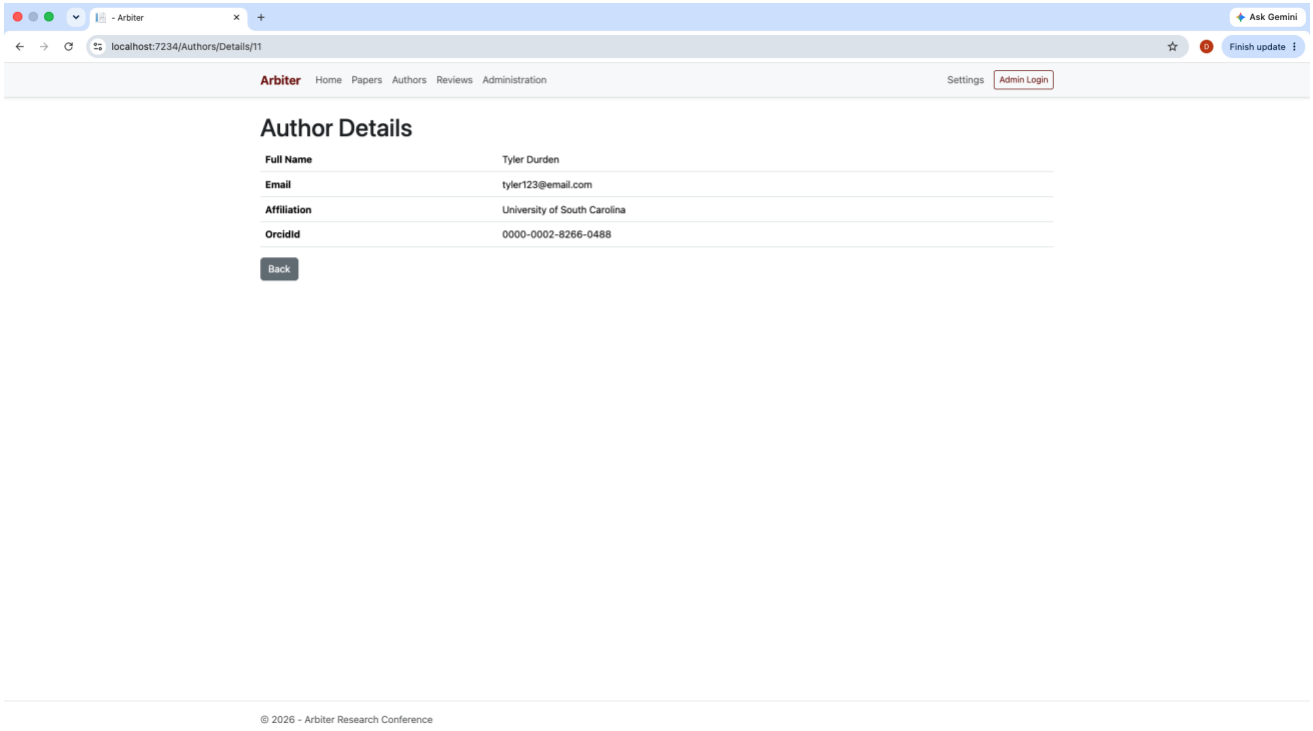


Figure 3. Read Author

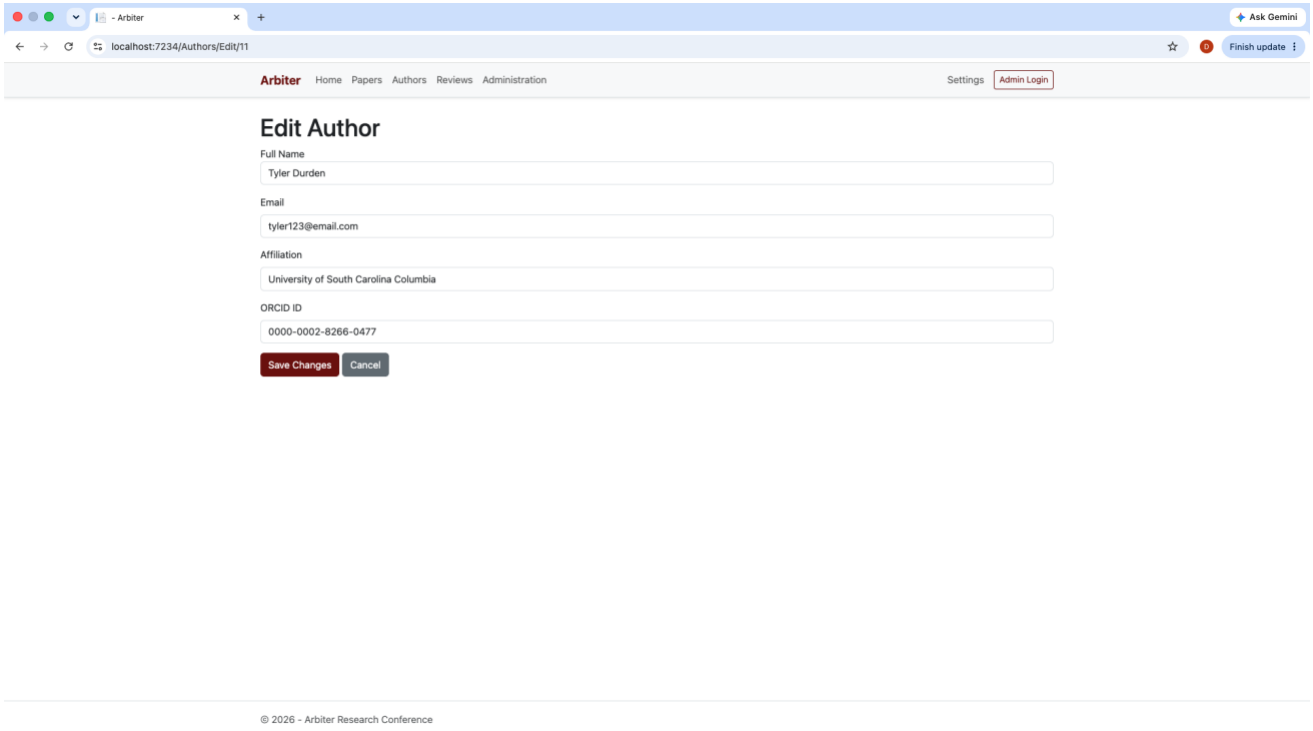


Figure 4. Update Author

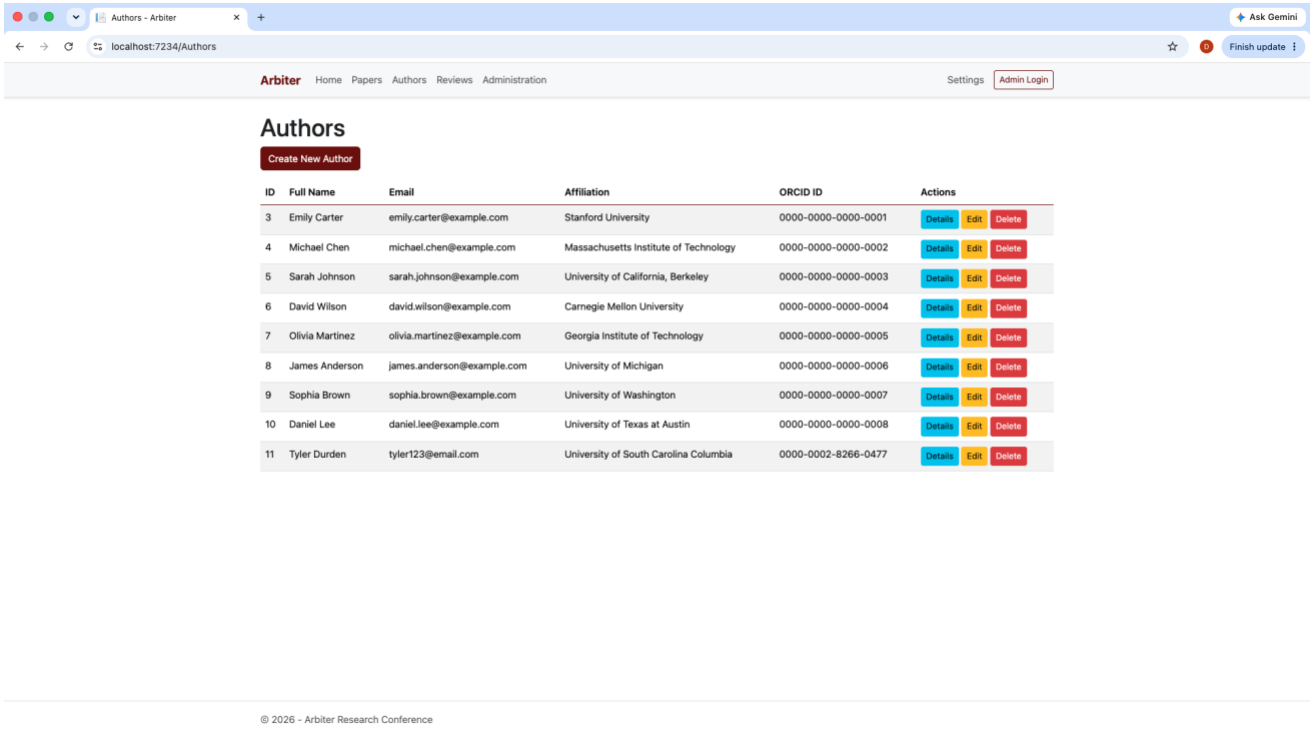


Figure 5. Update Successful

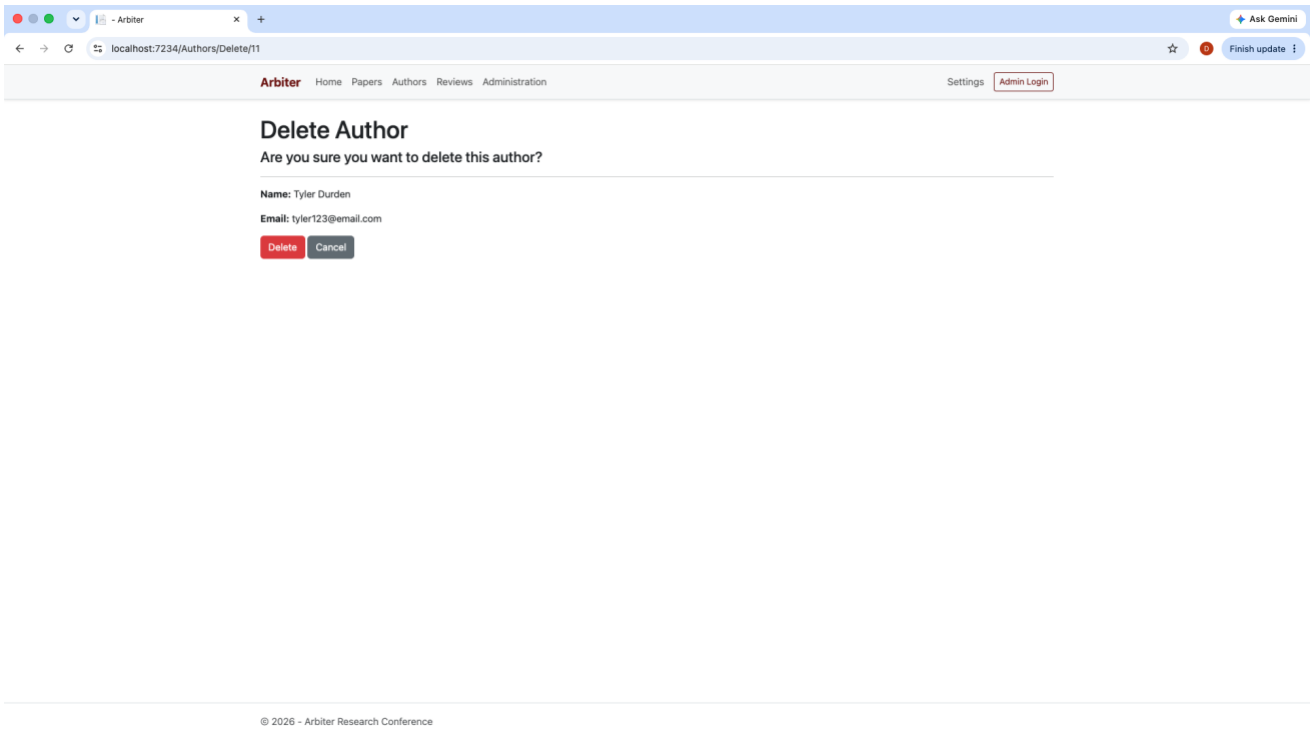


Figure 6. Delete Author

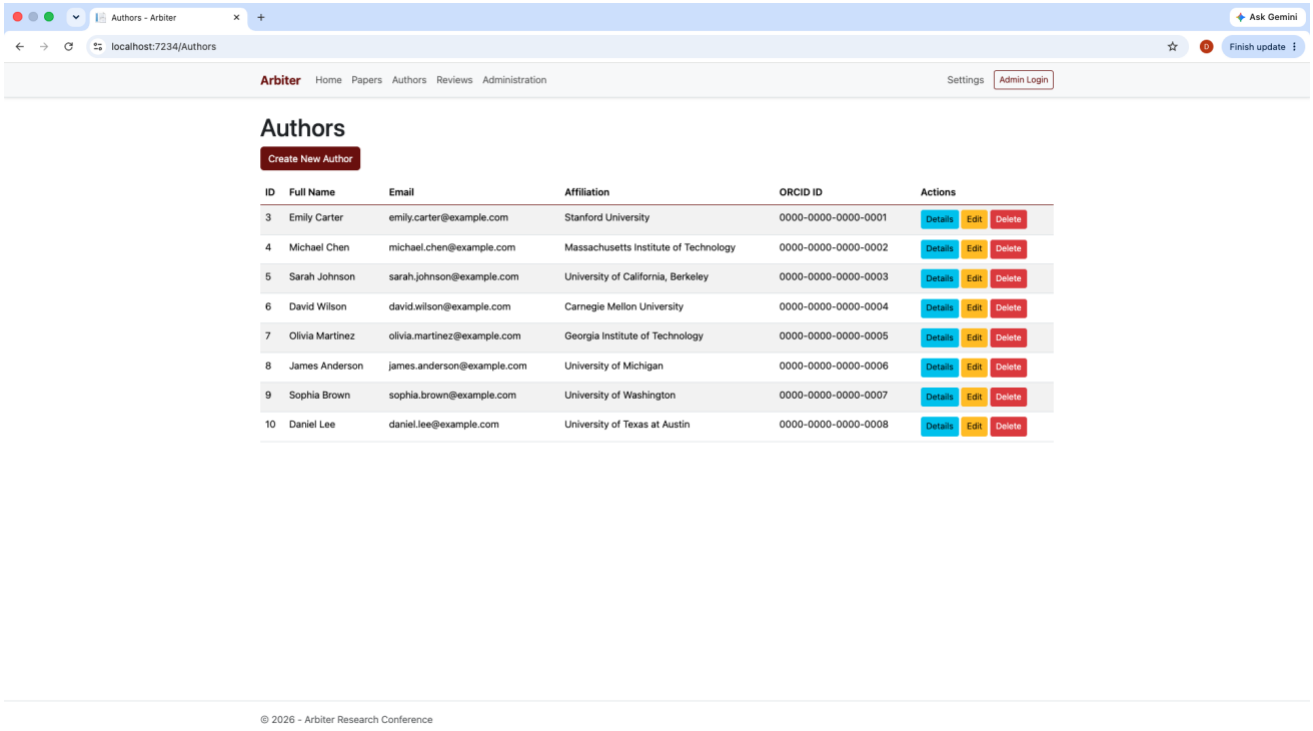


Figure 7. Delete Successful

Review CRUD

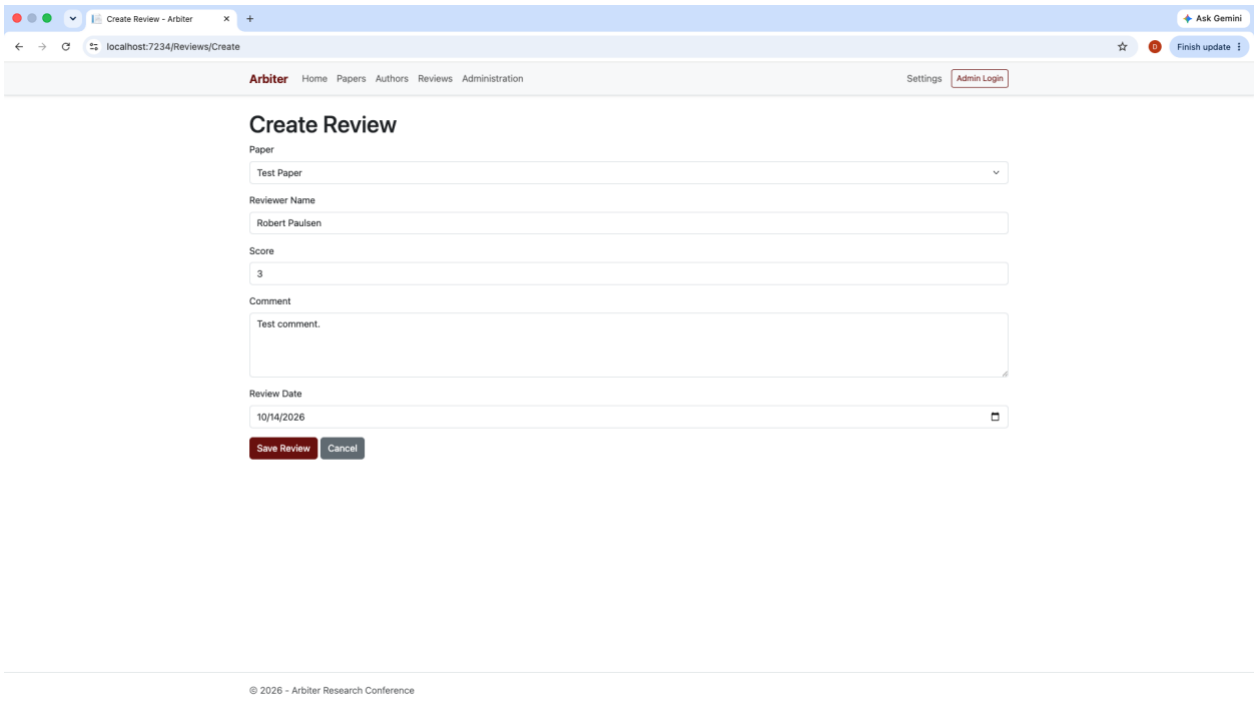


Figure 1. Create Review

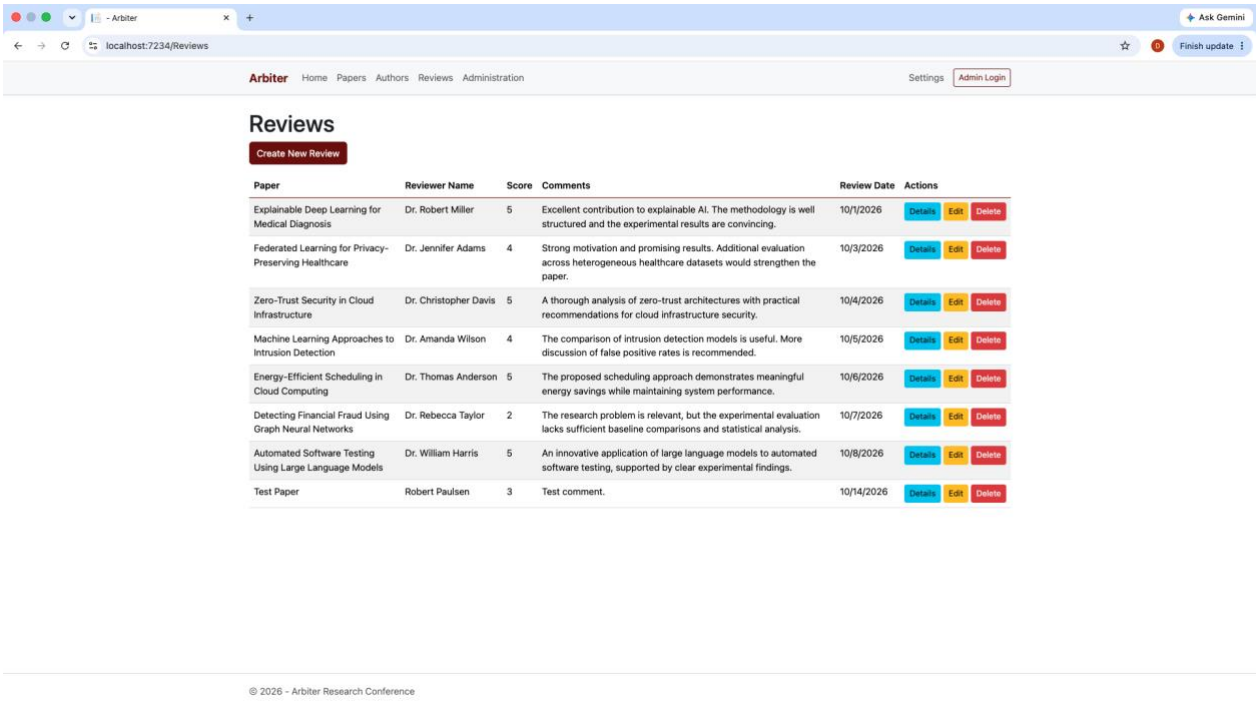


Figure 2. Create Successful

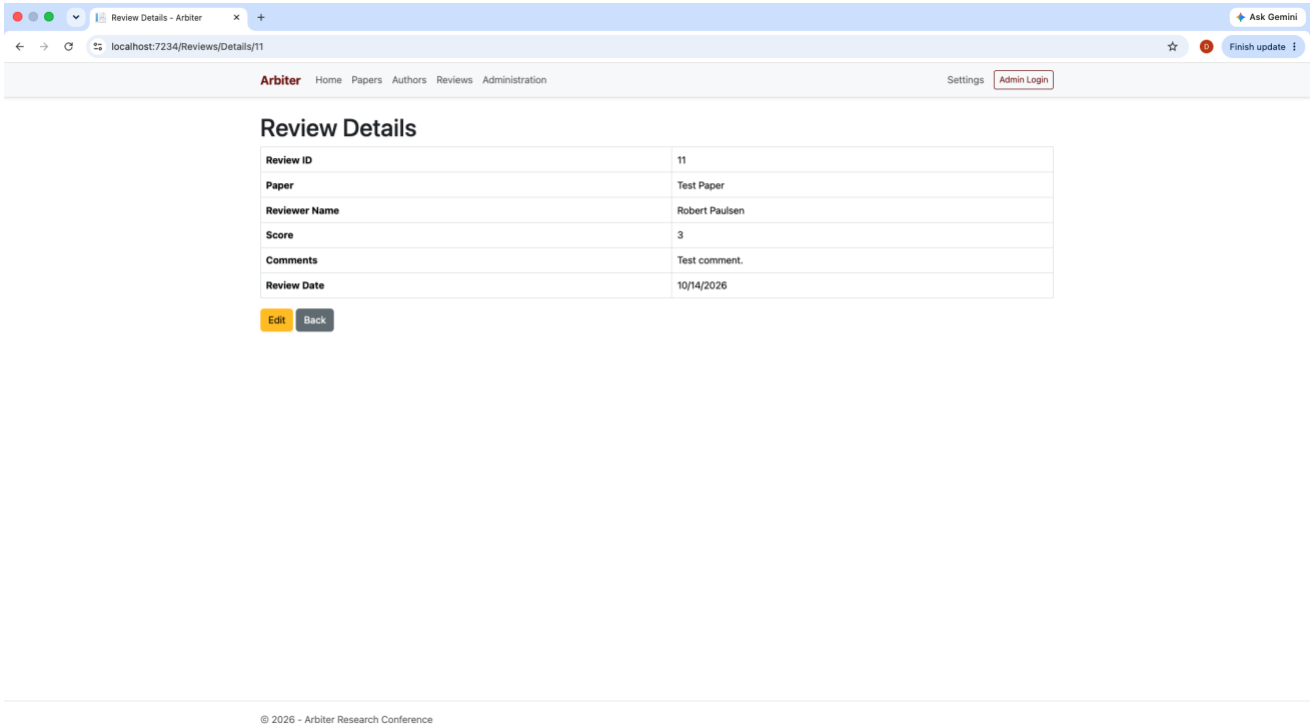


Figure 3. Read Review

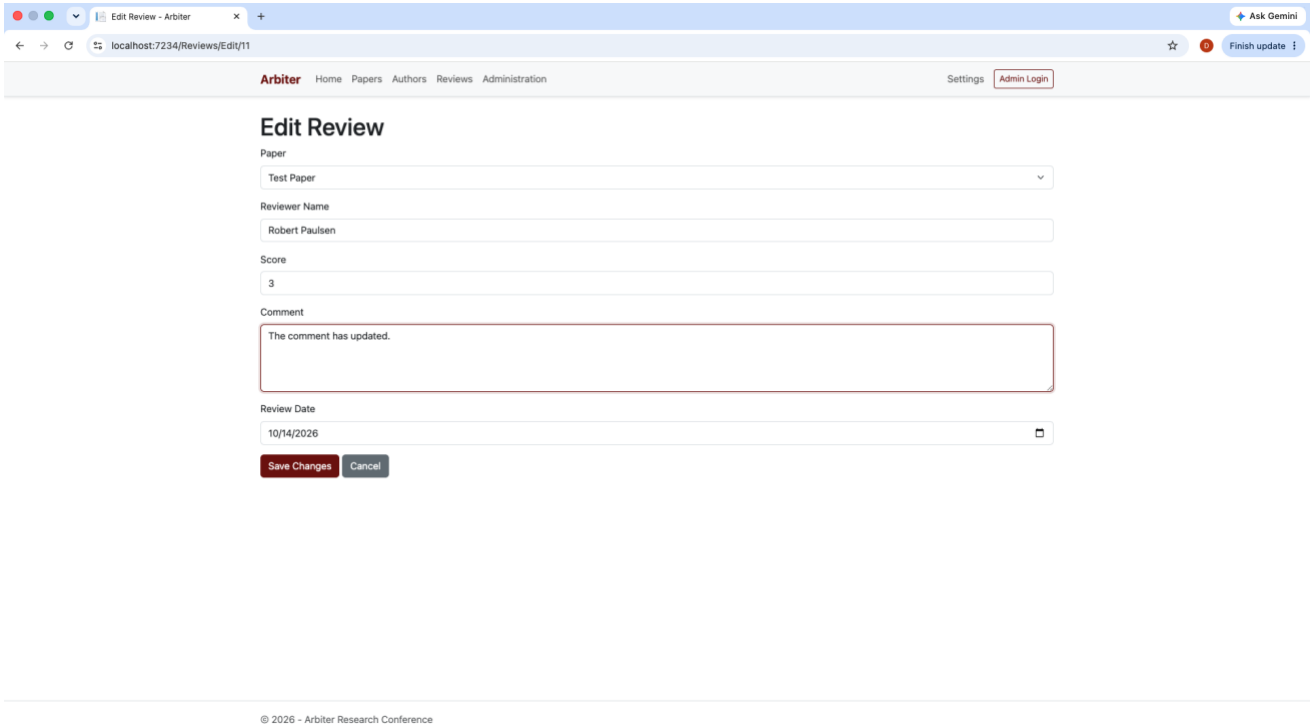


Figure 4. Update Review

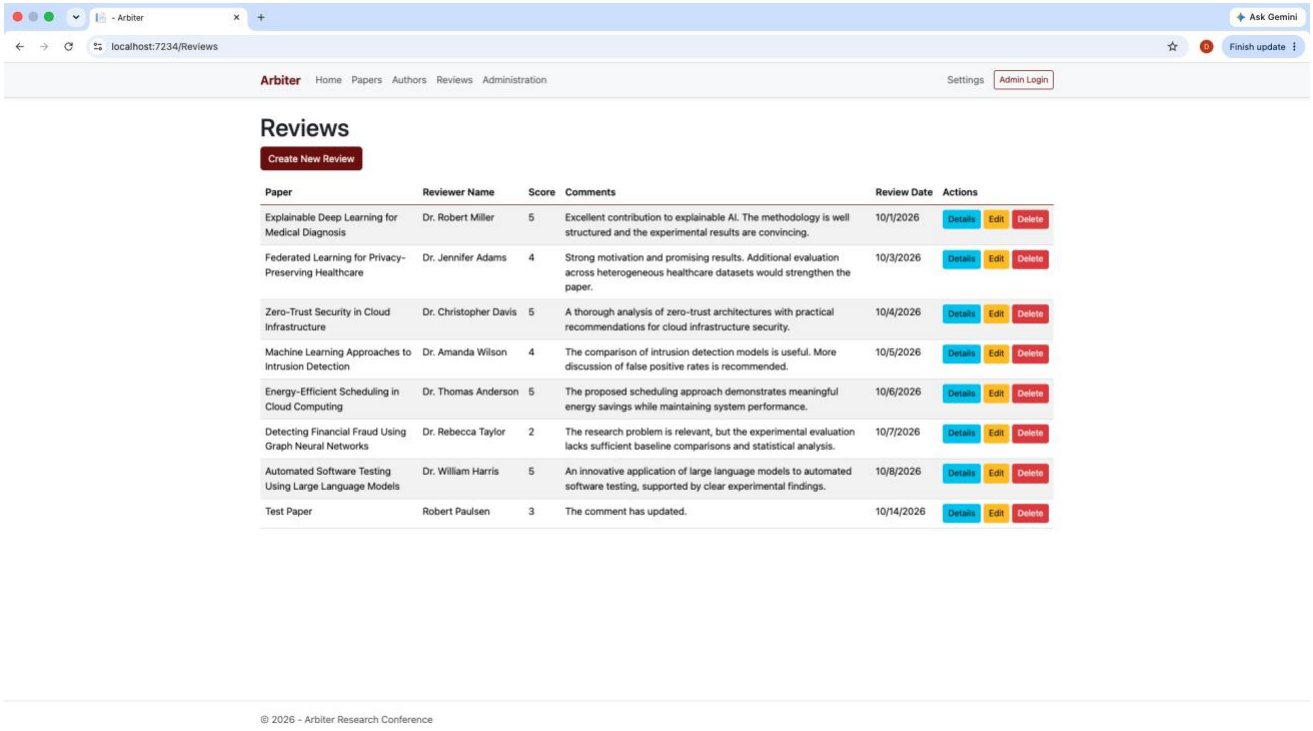


Figure 5. Update Successful

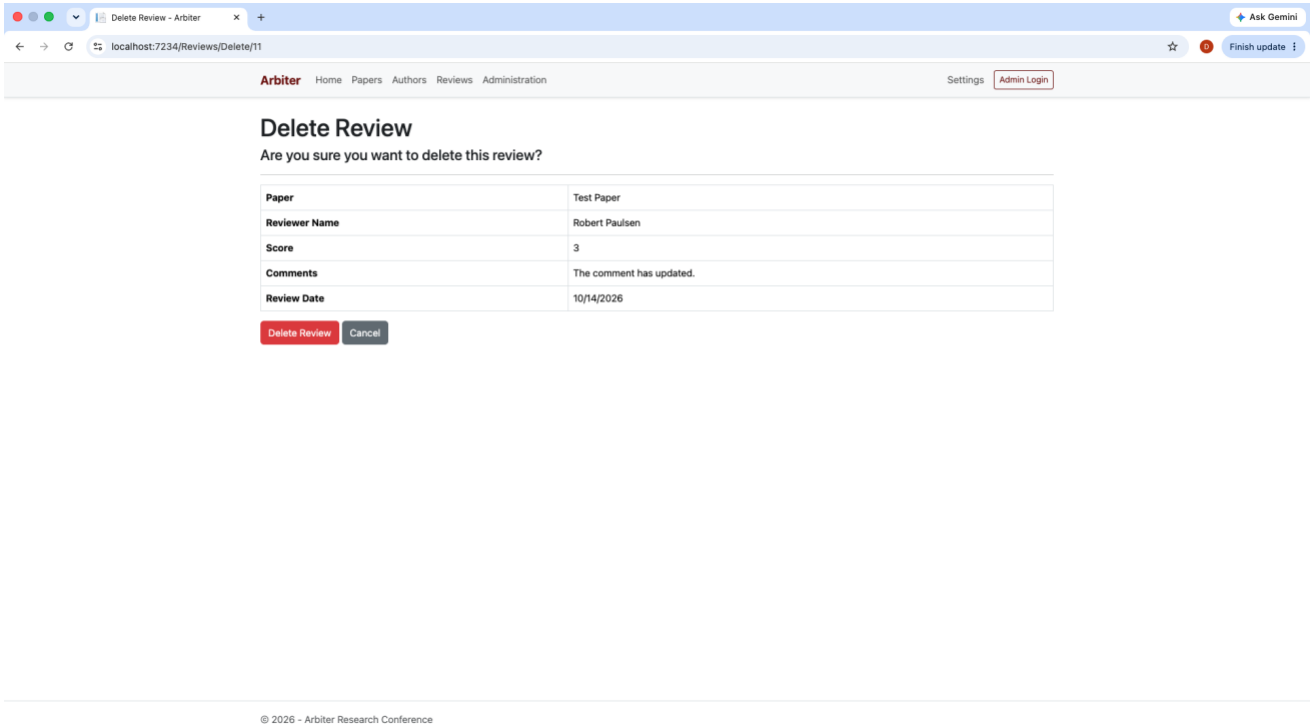


Figure 6. Delete Review

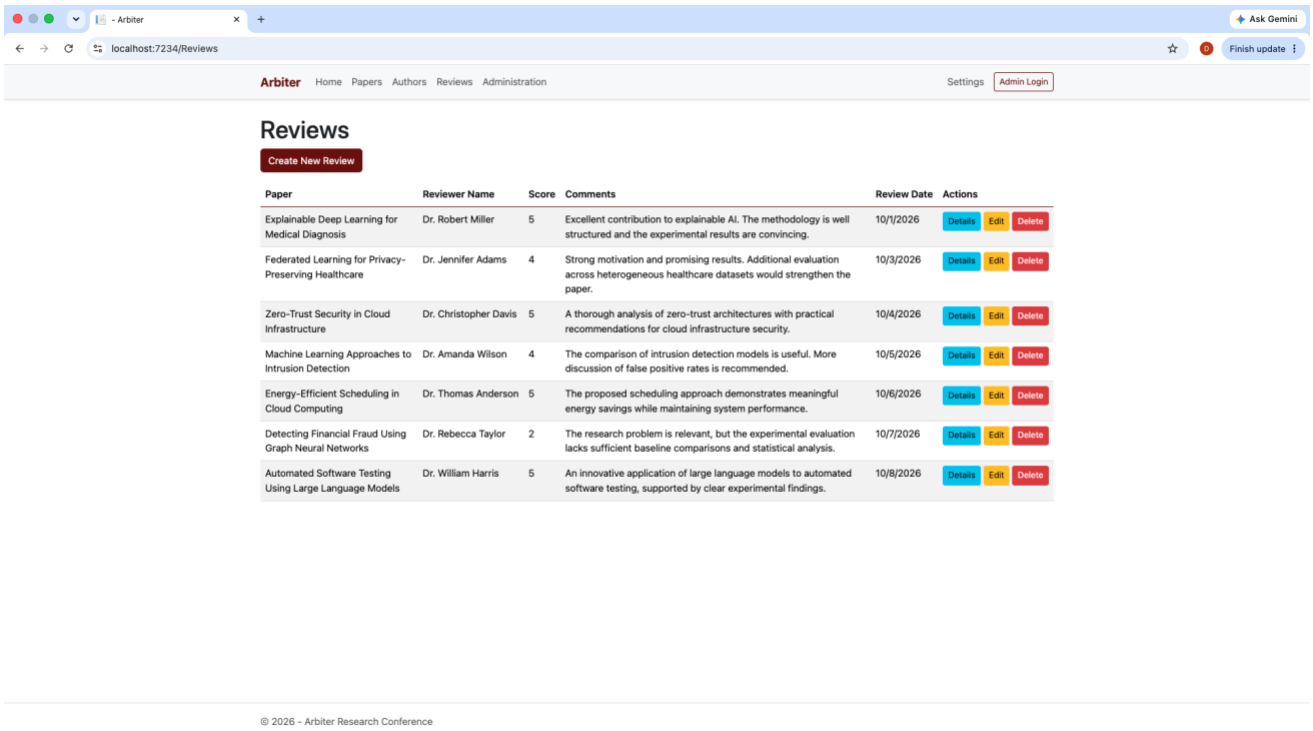


Figure 7. Delete Successful

Administration

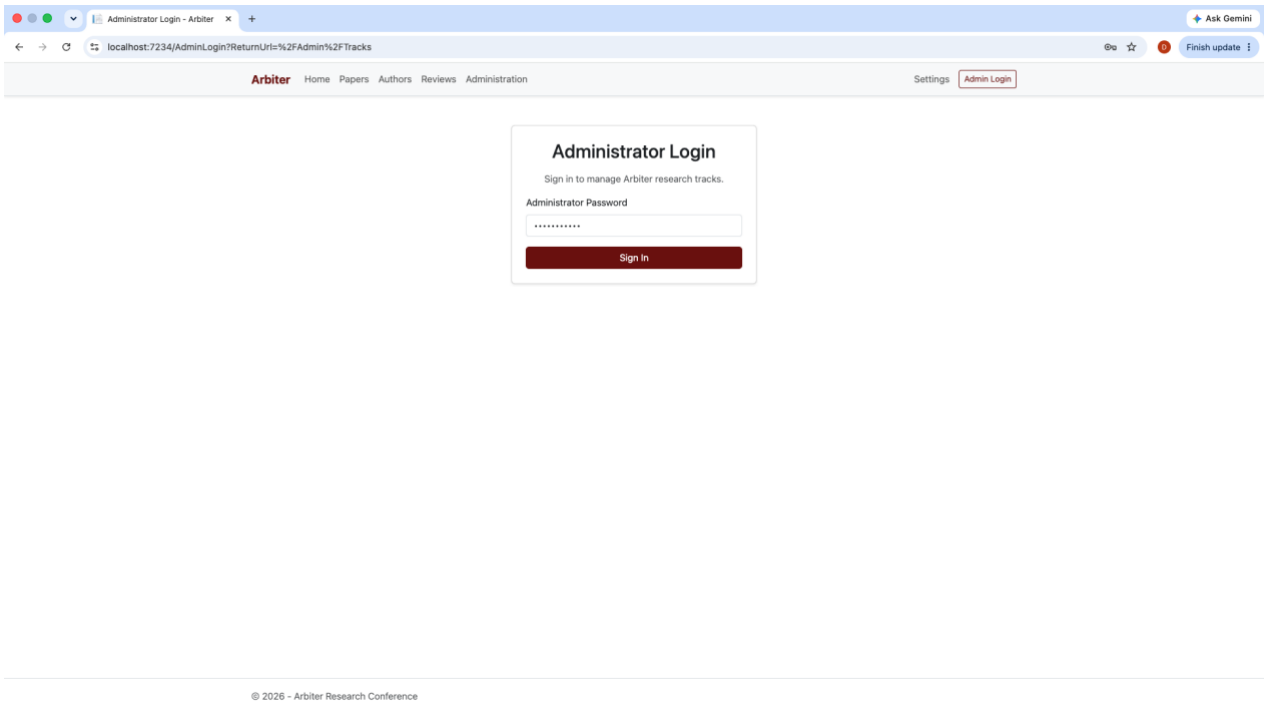


Figure 1. Admin login

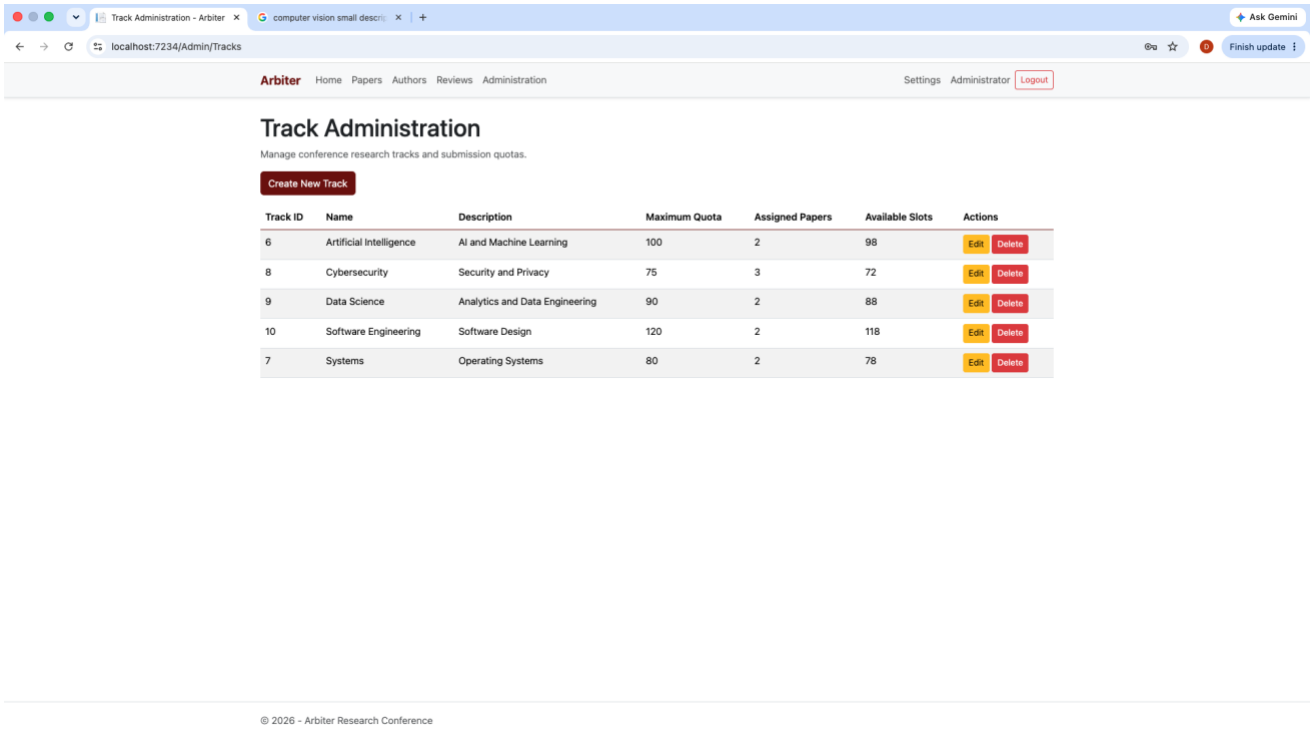


Figure 2. Admin page

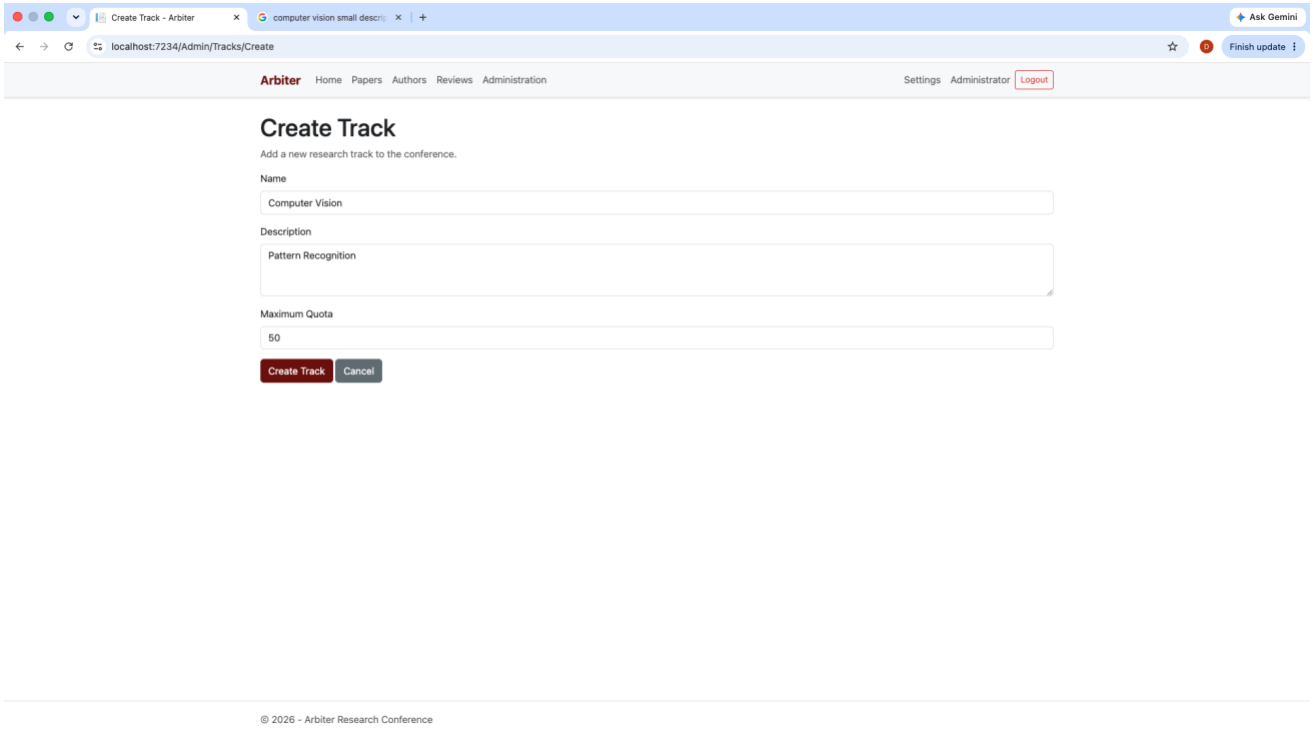


Figure 3. Admin Create

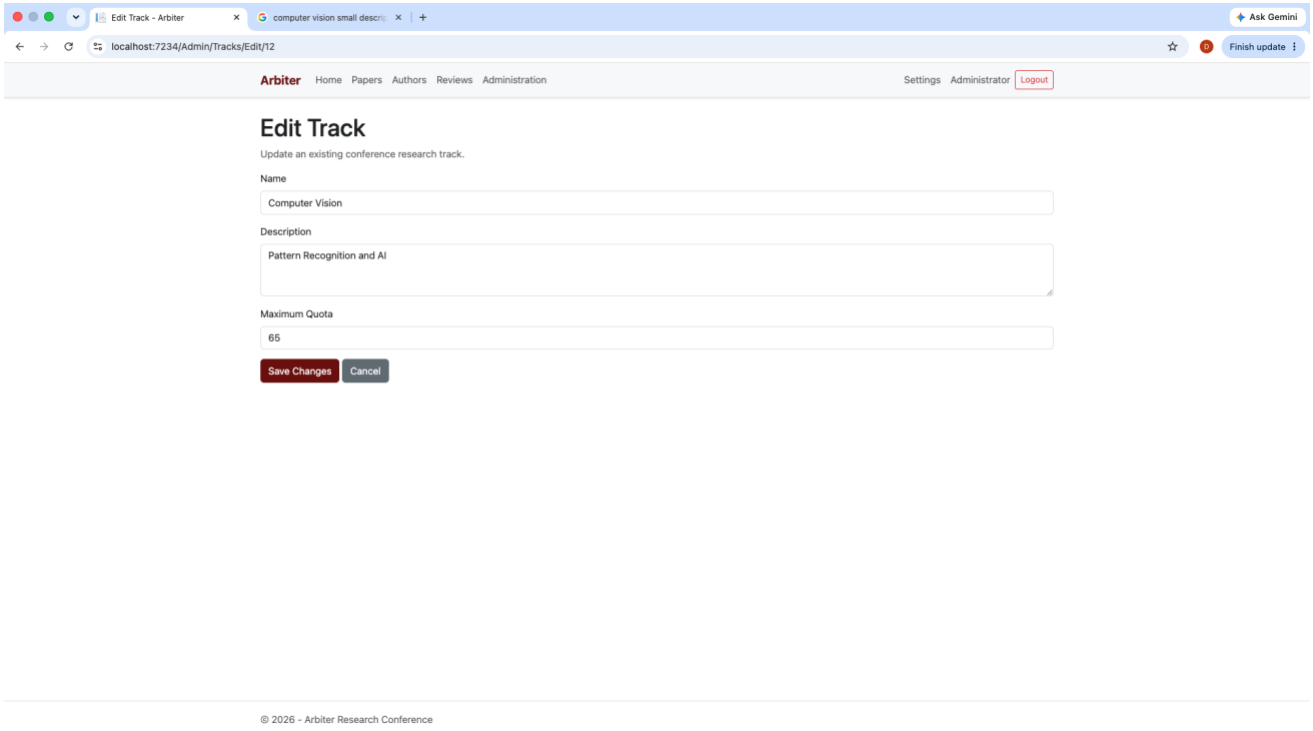


Figure 4. Admin Update

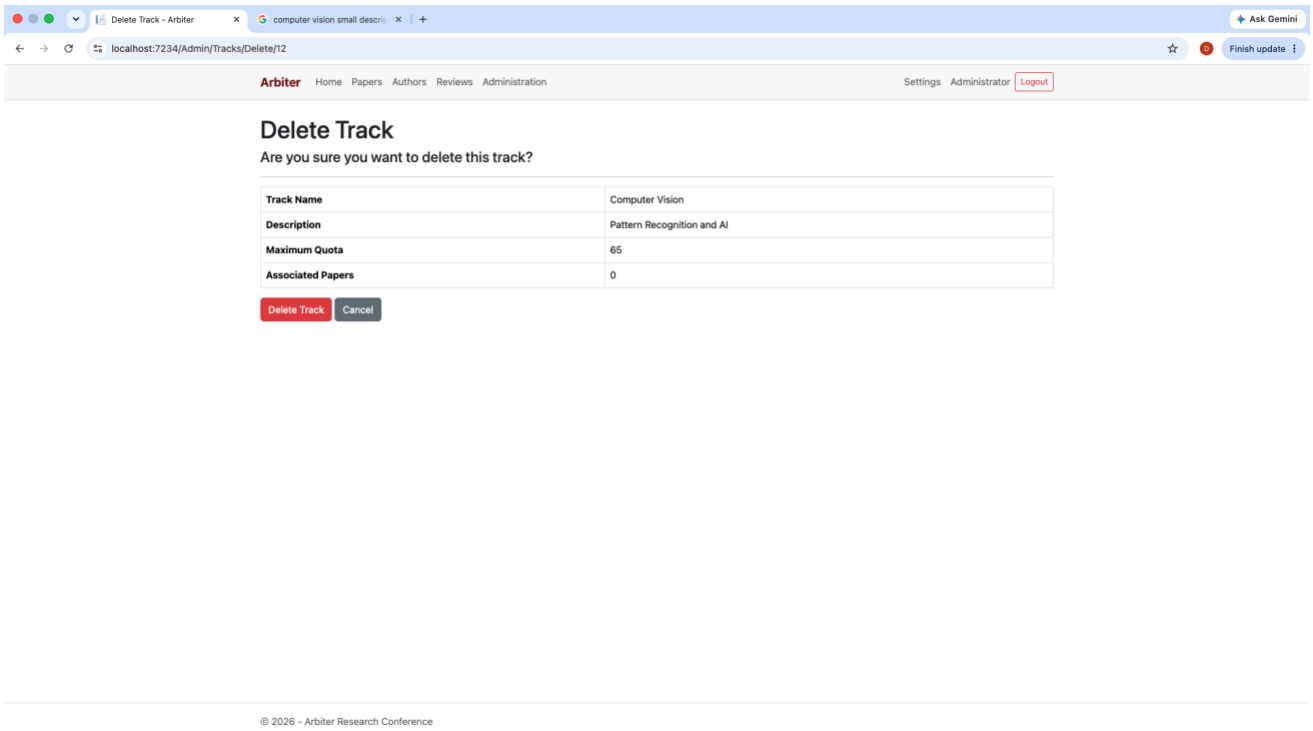


Figure 5. Admin Delete

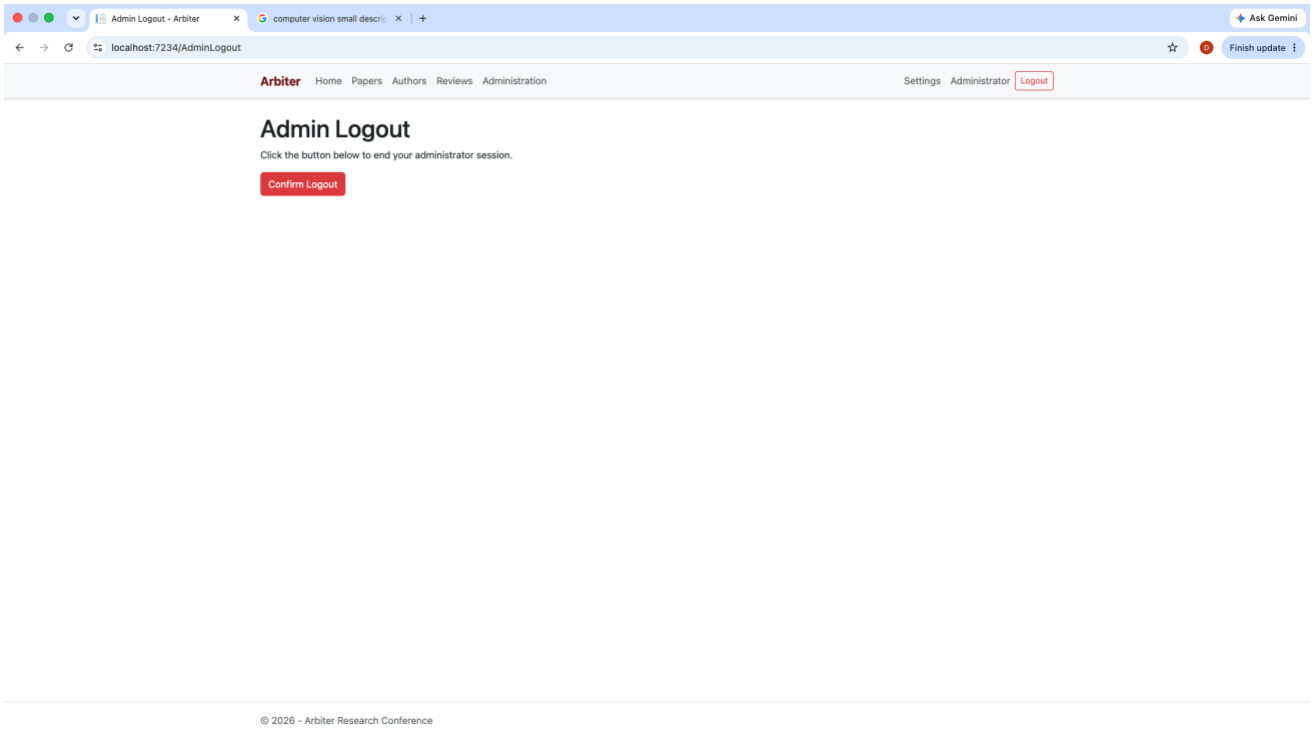
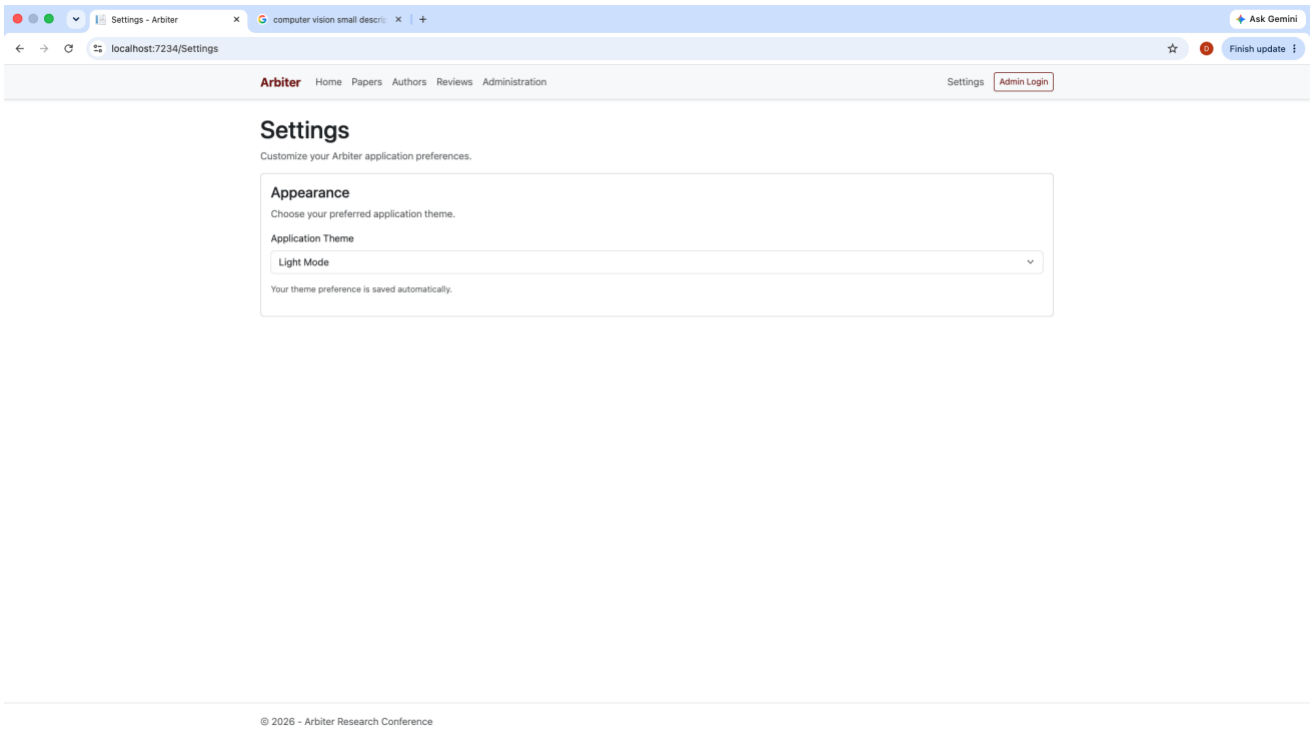
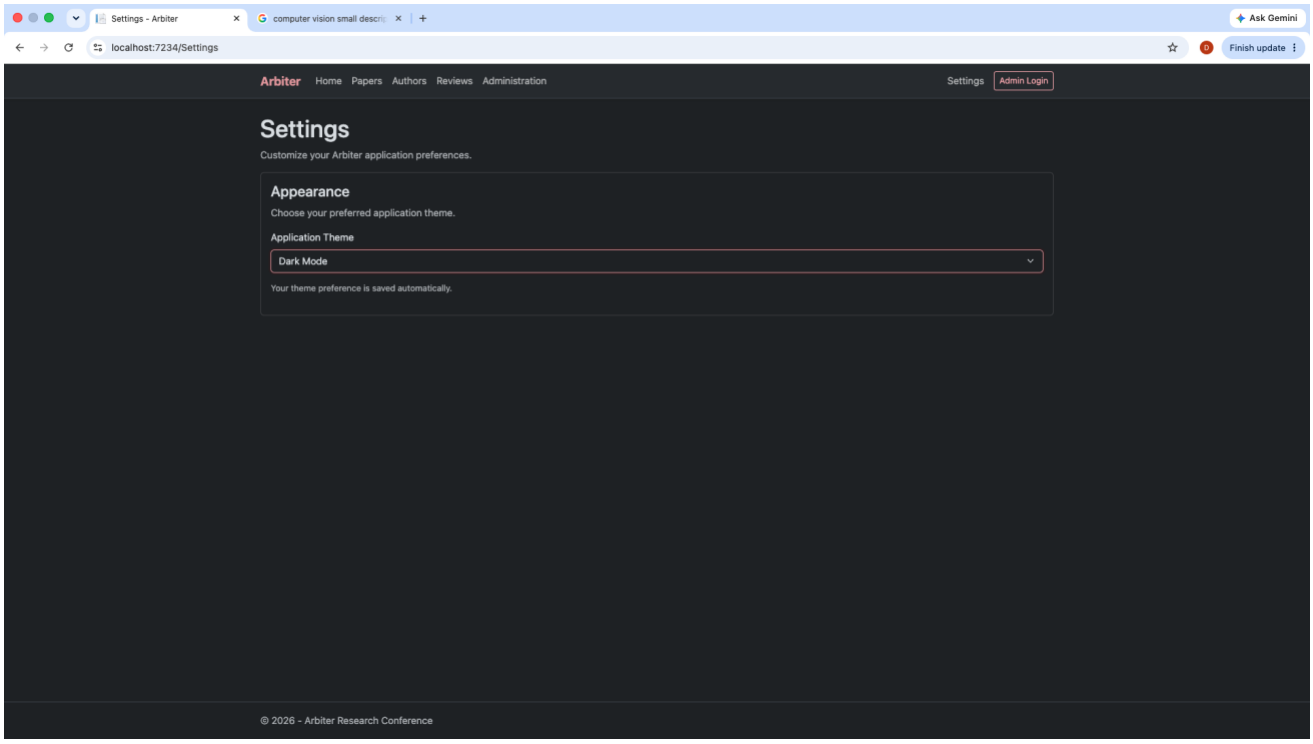
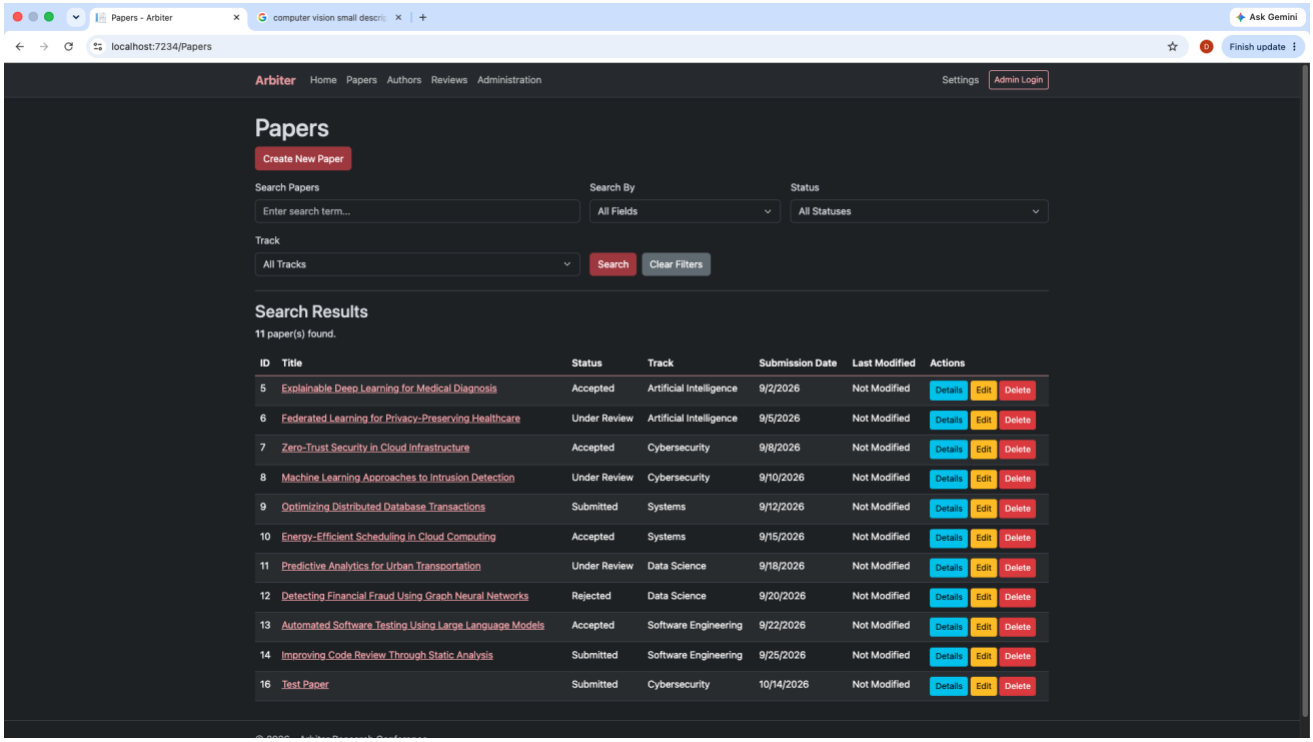


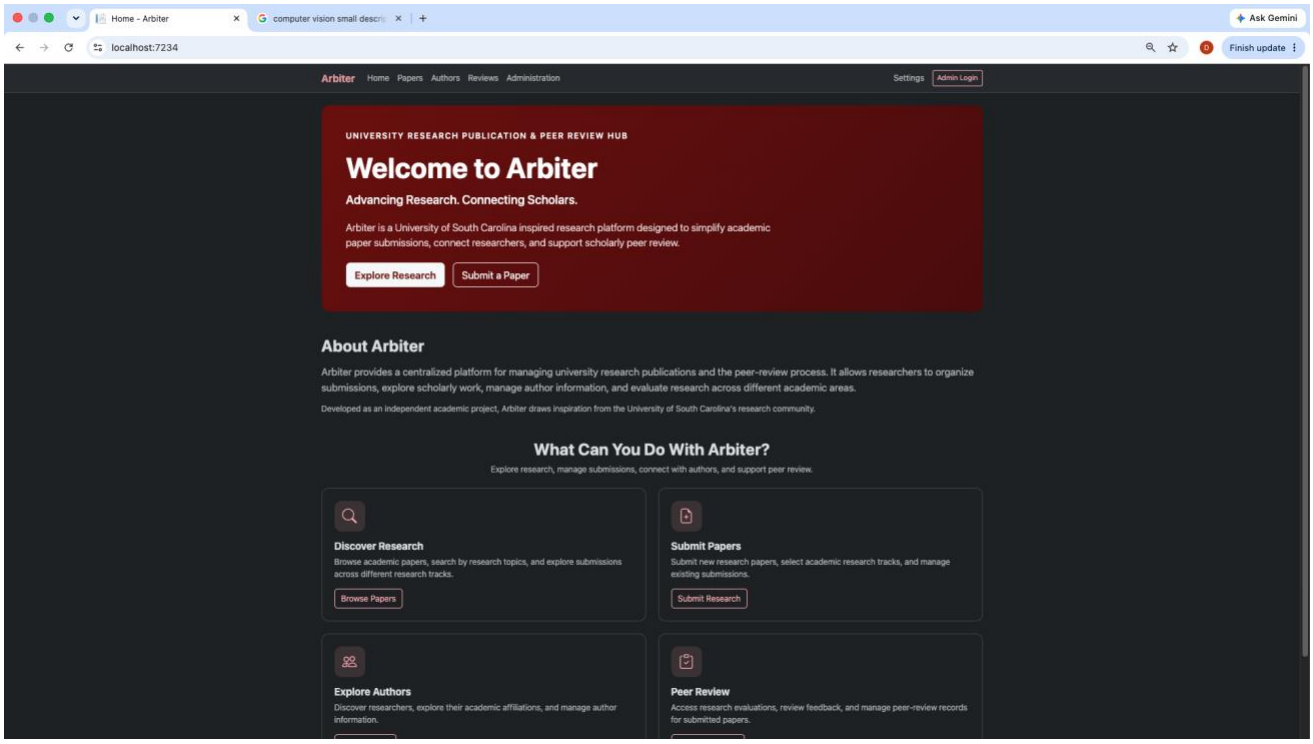
Figure 6. Admin logout

Settings: Light & Dark Theme






Papers - Arbiter
localhost:7234/Papers
Ask Gemini
Finish update
Arbiter
Home
Papers
Authors
Reviews
Administration
Settings
Admin Login
Papers
Create New Paper
Search Papers
Enter search term...
Search By
All Fields
Status
All Statuses
Track
All Tracks
Search
Clear Filters
Search Results
11 paper(s) found.
ID
Title
Status
Track
Submission Date
Last Modified
Actions
5 Explainable Deep Learning for Medical Diagnosis Accepted Artificial Intelligence 9/2/2026 Not Modified Details Edit Delete
6 Federated Learning for Privacy-Preserving Healthcare Under Review Artificial Intelligence 9/5/2026 Not Modified Details Edit Delete
7 Zero-Trust Security in Cloud Infrastructure Accepted Cybersecurity 9/8/2026 Not Modified Details Edit Delete
8 Machine Learning Approaches to Intrusion Detection Under Review Cybersecurity 9/10/2026 Not Modified Details Edit Delete
9 Optimizing Distributed Database Transactions Submitted Systems 9/12/2026 Not Modified Details Edit Delete
10 Energy-Efficient Scheduling in Cloud Computing Accepted Systems 9/15/2026 Not Modified Details Edit Delete
11 Predictive Analytics for Urban Transportation Under Review Data Science 9/18/2026 Not Modified Details Edit Delete
12 Detecting Financial Fraud Using Graph Neural Networks Rejected Data Science 9/20/2026 Not Modified Details Edit Delete
13 Automated Software Testing Using Large Language Models Accepted Software Engineering 9/22/2026 Not Modified Details Edit Delete
14 Improving Code Review Through Static Analysis Submitted Software Engineering 9/25/2026 Not Modified Details Edit Delete
16 Test Paper Submitted Cybersecurity 10/14/2026 Not Modified Details Edit Delete


Home - Arbiter
localhost:7234
Ask Gemini
Finish update
Arbiter
Home
Papers
Authors
Reviews
Administration
Settings
Admin Login
UNIVERSITY RESEARCH PUBLICATION & PEER REVIEW HUB
Welcome to Arbiter
Advancing Research. Connecting Scholars.
Arbiter is a University of South Carolina inspired research platform designed to simplify academic paper submissions, connect researchers, and support scholarly peer review.
Explore Research
Submit a Paper
About Arbiter
Arbiter provides a centralized platform for managing university research publications and the peer-review process. It allows researchers to organize submissions, explore scholarly work, manage author information, and evaluate research across different academic areas.
Developed as an independent academic project, Arbiter draws inspiration from the University of South Carolina's research community.
What Can You Do With Arbiter?
Explore research, manage submissions, connect with authors, and support peer review.
Discover Research
Browse academic papers, search by research topics, and explore submissions across different research tracks.
Browse Papers
Submit Papers
Submit new research papers, select academic research tracks, and manage existing submissions.
Submit Research
Explore Authors
Discover researchers, explore their academic affiliations, and manage author information.
Peer Review
Access research evaluations, review feedback, and manage peer-review records for submitted papers.